

Adaptive Parameter-Efficient Federated Fine-Tuning on Heterogeneous Devices

Jun Liu , Yunming Liao , Hongli Xu , *Member, IEEE*, Yang Xu , *Member, IEEE*,
Jianchun Liu , *Member, IEEE*, and Chen Qian , *Senior Member, IEEE*

Abstract—Federated fine-tuning (FedFT) has been proposed to fine-tune the pre-trained language models in a distributed manner. However, there are two critical challenges for efficient FedFT in practical applications, i.e., resource constraints and system heterogeneity. Existing works rely on parameter-efficient fine-tuning methods, e.g., low-rank adaptation (LoRA)¹, but with major limitations. Herein, based on the inherent characteristics of FedFT, we observe that LoRA layers with higher ranks added close to the output help to save resource consumption while achieving comparable fine-tuning performance. Then we propose a novel LoRA-based FedFT framework, termed LEGEND, which faces the difficulty of determining the number of LoRA layers (called, LoRA depth) and the rank of each LoRA layer (called, rank distribution). We analyze the coupled relationship between LoRA depth and rank distribution, and design an efficient LoRA configuration algorithm for heterogeneous devices, thereby promoting fine-tuning efficiency. Extensive experiments are conducted on a physical platform with 80 commercial devices. The results show that LEGEND can achieve a speedup of $1.5\text{--}2.8\times$ and save communication costs by about 42.3% when achieving the target accuracy, compared to the advanced solutions.

Index Terms—Low-rank adaptation, natural language processing, federated fine-tuning.

I. INTRODUCTION

THE emergence of transformer [1] and its variants [2], [3], [4] has catalyzed significant advancements in natural language processing (NLP), unveiling the immense potential for deploying NLP models¹ on various devices. The existing NLP paradigm encompasses two stages, i.e., pre-training and fine-tuning [3], [5]. Specifically, the language model (LM) is first pre-trained on a large corpus to learn general features and patterns. Subsequently, the LM is further fine-tuned on

domain-specific data generated on devices to enhance the model performance for a specific task. However, it is infeasible to collect enough domain-specific data from devices for centralized fine-tuning due to data privacy [6], [7], [8], [9]. To fully utilize the massive data on devices, federated fine-tuning (FedFT) has been proposed to perform fine-tuning in a distributed manner [7]. In the typical FedFT framework, e.g., FedNLP [7], participating devices periodically fine-tune the LMs on their local data, and push the local LMs to the parameter server (PS) for global aggregation without exposing their raw data. The fine-tuning procedure is repeated for multiple rounds until the LM converges or reaches the target accuracy [6], [10], [11]. FedFT not only protects individual privacy but also fully utilizes plenty of computing resources on devices to enhance the fine-tuning performance of the LMs.

Challenges of FedFT: Although FedFT has demonstrated its advantages, it still faces the following two challenges in practical applications: (1) *Resource constraints*. Many devices such as smartphones and in-vehicle devices, typically have limited resources (e.g., memory, computing power), which are orders of magnitude weaker than cloud servers [12], [13], [14]. However, existing LMs, e.g., Llama [15], typically involve billions of parameters, requiring substantial computing power for fine-tuning [16], while resource-constrained devices always lead to very slow fine-tuning rates. (2) *System heterogeneity*. The devices commonly possess varying computing capabilities (e.g., CPU frequency) or communication capabilities (e.g., bandwidth), which could differ from each other by more than tenfold [17], [18], [19]. Specifically, there are huge gaps of computing/communication capabilities among different types of devices, and even among devices of the same type with diverse configurations (e.g., smartphones, laptops) [13]. Due to system heterogeneity, fast devices should be forced to wait for slow ones, leading to prolonged waiting time and poor fine-tuning efficiency.

Status Quo and Limitations: To handle the issue of resource constraints, existing works rely on parameter-efficient fine-tuning methods [20], e.g., Adapter [21] and low-rank adaptation (LoRA) [16], which only fine-tune additional lightweight parameters (typically less than 1%). The Adapter method inserts additional blocks between two continuous transformer layers and only updates the parameters of the inserted blocks to achieve efficient fine-tuning [21]. For example, Cai et al. [11] first apply Adapter in FedFT and propose FedAdapter, which dynamically searches for the optimal Adapter structure to improve the

Received 9 March 2025; revised 25 May 2025; accepted 3 July 2025. Date of publication 7 July 2025; date of current version 3 October 2025. This work was supported by the National Science Foundation of China (NSFC) under Grant 62132019, Grant 62472401, and Grant 624B2136. Recommended for acceptance by S. Mazuelas. (*Corresponding author: Hongli Xu.*)

Jun Liu, Yunming Liao, Hongli Xu, Yang Xu, and Jianchun Liu are with the School of Computer Science and Technology, University of Science and Technology of China, Hefei 230027, China, and also with Suzhou Institute for Advanced Research, University of Science and Technology of China, Suzhou 215123, China (e-mail: lj18090759057@mail.ustc.edu.cn; ymliao98@mail.ustc.edu.cn; xuhongli@ustc.edu.cn; xuyangcs@ustc.edu.cn; jcliu17@ustc.edu.cn).

Chen Qian is with the Department of Computer Science and Engineering, Jack Baskin School of Engineering, University of California Santa Cruz, Santa Cruz, CA 95064 USA (e-mail: cqian12@ucsc.edu).

Digital Object Identifier 10.1109/TMC.2025.3586644

¹Note that LoRA in this context refers to not the long-range radio communication technique, but the deep learning fine-tuning technique.

fine-tuning efficiency. However, the Adapter method inevitably brings additional inference latency, potentially resulting in up to a 30% latency increase [16], [22], [23], which is often unacceptable in practical applications, e.g., real-time sentiment analysis [24] and real-time news categorization [25]. In addition, none of these methods have been tested under real wireless network environments with heterogeneous devices.

To avoid the extra inference latency, LoRA [16] has been proposed to freeze the pre-trained LM and add trainable bypass low-rank matrices (i.e., *LoRA layers*) to the transformer layers, in which only the LoRA layers would be updated during fine-tuning. The vanilla LoRA adds the LoRA layers with the *same* rank (i.e., the dimension of the bypass low-rank matrices) to *all* transformer layers [26]. To explore the potential of LoRA in FedFT, Zhang et al. [20] propose FedLoRA and verify the efficiency of FedFT with vanilla LoRA through experiments. Upon FedLoRA, Cho et al. [27] propose HetLoRA, in which each device adds LoRA layers to all transformer layers with a diverse and appropriate LoRA rank to deal with system heterogeneity. However, due to the rank mismatch of all LoRA layers on different devices, it is difficult to aggregate these layers, resulting in poor fine-tuning performance. In a nutshell, existing works simply add LoRA layers with the uniform rank distribution for all transformer layers, which still requires substantial computing and communication resources, resulting in slow fine-tuning speeds on weak devices. Moreover, system heterogeneity further leads to low fine-tuning efficiency or poor fine-tuning performance [28], [29], [30]. They do not address the two above challenges.

Overview of the Proposed Approach: According to the pre-test in Section II, adding the LoRA layers with different LoRA configurations (e.g., LoRA depth and rank distribution) significantly impacts the fine-tuning performance and resource consumption, where LoRA depth denotes the number of the continuous LoRA layers from the output. Adding LoRA layers with higher ranks to transformer layers close to the output helps to save resource consumption while achieving comparable fine-tuning performance. Based on the insight, we propose a novel FedFT framework with adaptive LoRA depth and rank distribution on **heterogeneous devices**, termed LEGEND, to deal with resource constraint and system heterogeneity. Our unique findings include: 1) large LoRA depth with high rank helps to achieve superior fine-tuning performance but incurs significant resource consumption, leading to a slow convergence rate on devices with limited resources; 2) Small LoRA depth with low ranks reduces the fine-tuning overhead on devices while resulting in poor fine-tuning performance or even failure to converge. Therefore, it is necessary yet challenging to simultaneously determine the appropriate LoRA depth with reasonable rank distribution for heterogeneous devices, so as to well balance the trade-off between fine-tuning performance and resource consumption. To our knowledge, this is the first study of federated learning based NLP in a real wireless testbed with heterogeneous devices. The main contributions of this paper can be summarized as:

- To address the challenges of resource constraints and system heterogeneity, we propose an efficient LoRA-based FedFT framework, called LEGEND, by reviewing the inherent characteristics of FedFT.

- We analyze the joint influence of the LoRA depth and rank distribution on fine-tuning performance and obtain their coupled relationship. Upon this, we develop an efficient algorithm to carefully determine the appropriate LoRA depths with a reasonable rank distribution across all selected LoRA layers for heterogeneous devices to improve the fine-tuning performance.
- The performance of LEGEND is evaluated through a physical platform with 80 commercial devices that are connected via WiFi. The experimental results show that LEGEND provides a speedup of fine-tuning by about 1.5-2.8 $\times$ and saves communication costs by about 42.3% when achieving the target accuracy, compared to existing solutions.

II. BACKGROUND AND MOTIVATION

A. Federated Fine-Tuning LMs With LoRA

Language Models: From design to deployment, training transformer-based LMs typically involves two main stages: pre-training and fine-tuning [1], [7], [11]. In the first stage, the LMs are pre-trained on the large-scale corpus, e.g., Wikipedia [31], C4 [32], to learn the ubiquitous linguistic structure, which is independent of downstream tasks [11]. The pre-training stage demands massive computing resources, typically undertaken by large tech companies [3], [5], such as Google and Microsoft. Fine-tuning adapts the pre-trained LMs to various downstream tasks, such as text classification and text generation, which require substantial data to update the entire LM for a given task. However, fine-tuning the entire LM usually demands excessive resources, e.g., computing and communication resources, leading to slow fine-tuning speeds on resource-constraint devices [11], [33].

LoRA for LMs: Herein, low-rank adaptation (LoRA) adds trainable rank decomposition matrices to each transformer layer of the LM while freezing the pre-trained weights of the LM to improve fine-tuning efficiency [16]. For a pre-trained weight matrix $\mathcal{M} \in \mathbb{R}^{m \times q}$ (m and q are the dimension sizes of $\mathcal{M}$), LoRA injects low-rank decomposition $\Delta\mathcal{M} = \mathcal{B}\mathcal{A}$ as the trainable parameters. Note that, $\mathcal{B} \in \mathbb{R}^{m \times r}$ and $\mathcal{A} \in \mathbb{R}^{r \times q}$ are the project-down matrix and the project-up matrix respectively, where r denotes the rank of LoRA and is much smaller than both m and q . Formally, for a simple linear layer $y = \mathcal{M}x$ in the transformer layer, LoRA modifies the forward propagation as follows:

$$y = \mathcal{M}x + \mathcal{B}\mathcal{A}x \quad (1)$$

where $x \in \mathbb{R}^{q \times s}$ and $y \in \mathbb{R}^{m \times s}$ are the input tensors and the output tensors, respectively, and s is the sequence length (i.e., the number of tokens in a given sequence). As a generation of full fine-tuning [16], LoRA utilizes two bypass matrices (i.e., $\mathcal{B}$ and $\mathcal{A}$) with the low rank to perform fine-tuning without modifying the pre-trained weight matrix, thus LoRA can be applied to any transformer layers of the LMs. Fine-tuning LMs with LoRA can greatly reduce the number of trainable parameters (typically less than 1% [16], [20]) while maintaining satisfactory fine-tuning performance.

FedFT with LoRA: Considering a distributed system with one PS and n devices, FedFT is proposed to fine-tune the

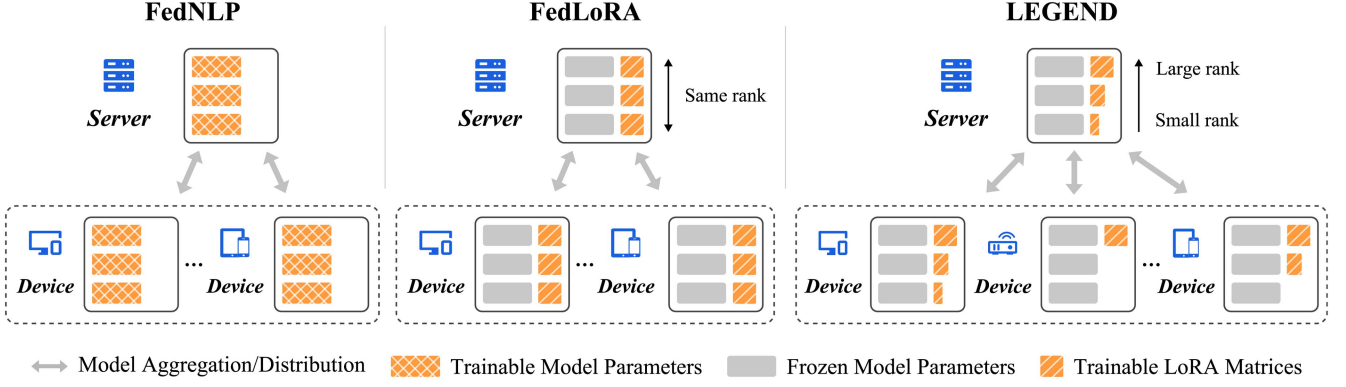


Fig. 1. Illustration of FedNLP, FedLoRA, and LEGEND. FedNLP (left) fine-tunes all parameters of the LM; FedLoRA (mid) applies the same LoRA configuration to all devices; LEGEND (right) applies different LoRA configurations (e.g., LoRA depth) to devices with heterogeneous capabilities.

LMs through a loose federation of devices. Naturally, LoRA is introduced into FedFT (e.g., FedLoRA) to reduce computing and communication overhead [20], [27]. The difference between FedFT (e.g., FedNLP) and FedLoRA is illustrated in Fig. 1. FedLoRA only exchanges lightweight LoRA layers $\tilde{\theta}$ instead of all cumbersome LMs $\bar{\theta}$ in FedFT. The goal of FedLoRA is to find the optimal model $\theta^* = \{\tilde{\theta}^*, \bar{\theta}\}$ minimizing the loss function $f(\theta)$ as:

$$\min_{\theta=\{\tilde{\theta}, \bar{\theta}\}} f(\theta) \triangleq \frac{1}{n} \sum_{i=1}^n f_i(\theta_i) \quad (2)$$

where $f_i(\theta_i) = \frac{1}{|\mathbb{D}_i|} \sum_{\xi_i \in \mathbb{D}_i} F_i(\theta_i; \xi_i)$ denotes the loss function of the local model θ_i on device i and $F_i(\theta_i; \xi_i)$ is the loss over data samples ξ_i on local dataset $\mathbb{D}_i$. To minimize the local objective function, device i only updates the LoRA layers through the gradient descent algorithm (e.g., AdamW [34]). The optimization procedure for updating the LoRA layers $\tilde{\theta}_i^h$ on device i at local step t in round h can be expressed as:

$$\tilde{\theta}_i^{h,t} = \tilde{\theta}_i^{h,t-1} - \eta \cdot \nabla f_i(\tilde{\theta}_i^{h,t-1}) \quad (3)$$

where η is the learning rate and $\nabla f_i(\tilde{\theta}_i^{h,t-1})$ is the gradient of the loss for LoRA layers $\tilde{\theta}_i^{h,t-1}$.

After local fine-tuning, all devices send the updated LoRA layers to the PS for global aggregation as follows:

$$\tilde{\theta}^{h+1} = \frac{1}{n} \sum_{i=1}^n \tilde{\theta}_i^h \quad (4)$$

After global aggregation, the PS distributes the latest LoRA layers to all devices and moves to the next fine-tuning round.

B. Importance of LoRA Position

Existing LoRA-based frameworks (e.g., FedLoRA [20] and HetLoRA [27]) typically add LoRA layers to all transformer layers, which requires the complete backpropagation process to update all LoRA layers [35] and results in a slow convergence rate for fine-tuning pre-trained LMs on resource-constrained devices. As illustrated in Fig. 2, we divide an LM (e.g., RoBERTa) into three parts representing different positions, i.e., shallow, medium and deep, respectively. Based on the backward

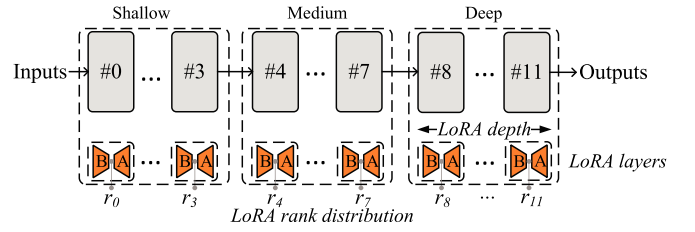


Fig. 2. Fine-tuning RoBERTa at different positions.

direction of the backpropagation process, fine-tuning the continuous LoRA layers added to the transformer layers at deep position is computationally efficient while achieving satisfactory fine-tuning performance. Wei et al. [36] provide rigorous theoretical insights into the convergence of partial LoRA layer fine-tuning. Besides, only the LoRA layers at deep position need to perform the backpropagation process and be transmitted to the PS, effectively reducing computing/communication overhead and speeding up the fine-tuning process. In addition, since pre-trained LMs have acquired powerful language understanding and generation capabilities during the pre-training stage, fine-tuning the added LoRA layers to partial transformer layers at deep position can essentially achieve comparable fine-tuning performance [7], [37], [38].

To demonstrate the importance of LoRA position, we conduct a set of experiments for federated fine-tuning RoBERTa [39] on SST-2 [40] with 10 devices (more experimental setup details in Section VII-A). We train a 12-layer RoBERTa with LoRA layers added to partial transformer layers at different positions (as illustrated in Fig. 2), including all layers (denoted as *Layers-A*), shallow layers {#0, #1, #2, #3} (denoted as *Layers-S*), medium layers {#4, #5, #6, #7} (denoted as *Layers-M*), and deep layers {#8, #9, #10, #11} (denoted as *Layers-D*). As shown in Fig. 3, we can derive the following conclusions:

1) Fine-tuning LoRA layers added to only the transformer layers at the deep position achieves comparable performance to vanilla LoRA. For example, as shown in Fig. 3(a), *Layers-D* achieves comparable final accuracy (i.e., 93.1%) to *Layers-A* (94.3%), with only fine-tuning a third of LoRA layers in *Layers-A* (i.e., less computing resource). This validates that deeper layers' capability to extract global features is more crucial

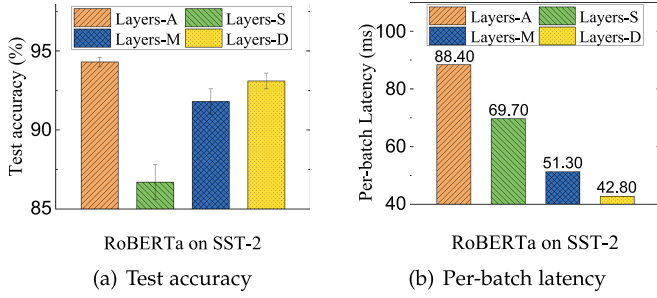


Fig. 3. The impact of LoRA position.

for adaptation tasks, making them ideal locations for LoRA fine-tuning. Moreover, compared with Layers-S and Layers-M, Layers-D improves the final model accuracy by 6.4% and 1.3%, respectively, demonstrating the effectiveness of targeting deep layers rather than shallow or medium ones.

2) Fine-tuning LoRA layers added to only the transformer layers at deep positions speeds up the fine-tuning process. For instance, by Fig. 3(b), Layers-D achieves 2.1 $\times$, 1.6 $\times$, and 1.2 $\times$ speedup compared to Layers-A, Layers-S, and Layers-M, respectively. This significant efficiency gain is attributed to Layers-D greatly shortening the backpropagation process from 12 transformer layers to only 4 transformer layers.

Thus, the LoRA position greatly affects fine-tuning performance and resource consumption, while adding LoRA layers to a fixed number of transformer layers at the deep position can save resource cost with satisfactory fine-tuning performance. Our findings are consistent with the relevant conclusions that layers at the deep position are more important than those at the shallow position [11], [12], [26].

C. Importance of LoRA Depth

In addition to the results of adding LoRA layers to partial transformer layers at deep position, we further explore the impact of the number of transformer layers at deep position to add extra LoRA layers (called LoRA depth) on fine-tuning performance. In general, LoRA depth is closely related to fine-tuning performance and resource consumption. Specifically, the larger LoRA depth means fine-tuning more layers at deep positions to enhance fine-tuning performance but leads to a longer backpropagation process (i.e., slower convergence rate). Although the smaller LoRA depth can shorten the backpropagation process to speed up the fine-tuning process, it constrains the number of tunable transformer layers and thus restricts the task-fitting capability of the LM. Therefore, it is critical and non-negligible to determine the appropriate LoRA depth for achieving the trade-off between fine-tuning performance and resource consumption.

To illustrate the impact of the LoRA depth, we conduct a set of experiments for fine-tuning RoBERTa on SST-2 with different LoRA depths (from 1 to 12) and rank of 8. We can draw the following conclusions from Fig. 4:

1) As LoRA depth increases, resource consumption (e.g., memory usage) increases almost linearly, resulting in a slow convergence rate. For example, by Fig. 4(a) and (b), with each additional LoRA layer, the per-batch latency increases

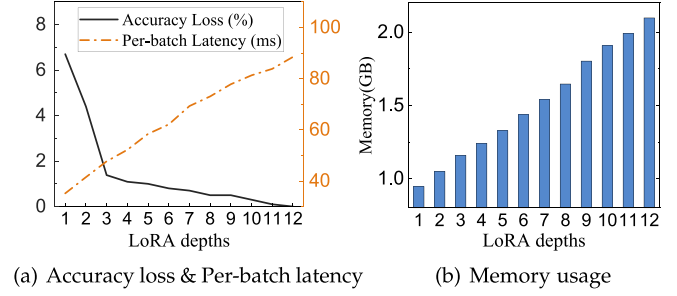


Fig. 4. The impact of LoRA depth.

by approximately 5 ms, and the memory usage increases by approximately 107 MB. Compared with LoRA depth of 1, fine-tuning RoBERTa with depth of 12 results in a 252% increase in per-batch latency and 221% growth in memory usage. This is because backpropagation only computes up to the selected fine-tuning depth, eliminating the need to store activation memory for shallower layers, thereby significantly reducing both computation time and memory consumption.

2) The fine-tuning performance improves with the increase of LoRA depth, but the magnitude of the improvement diminishes gradually. For instance, by Fig. 4(a), compared to the LoRA depth of 3, fine-tuning RoBERTa with LoRA depths of 6 and 9 exhibits improvements of 0.6% and 0.9% in accuracy, respectively. As LoRA depth increases from 1 to 3, the final model performance improves by 5.3%, but only 1.4% from LoRA depth of 3 to 12. This diminishing marginal return stems from the functional differentiation within the transformer architecture: higher layers primarily handle task-specific semantic understanding, while lower layers focus on basic feature extraction, thus extending fine-tuning beyond critical higher layers naturally yields decreasing benefits [18], [41].

By the results, carefully determining LoRA depth is critical to improve fine-tuning performance while saving computing and communication resources. Moreover, to deal with system heterogeneity, LEGEND assigns different LoRA depths for heterogeneous devices to adapt their capabilities. The devices with strong computing and communication capabilities are assigned with larger LoRA depths, while those with lower computing and communication capabilities are assigned with smaller LoRA depths, so as to reduce waiting time and further improve fine-tuning efficiency.

D. Importance of LoRA Rank Distribution

We further explore the impact of rank distribution on fine-tuning performance. In general, the rank of LoRA layer is closely related to the model capacity, and increasing rank is essential for better performance [42], [43]. Given a fixed total rank budget under resource constraints, strategically allocating higher ranks to task-relevant layers becomes crucial for maximizing model performance [26], [27]. Specifically, the higher ranks of the deep LoRA layers achieve better fine-tuning performance on a specific task, since the layers at deep position need to focus on higher semantic levels and more contextual information [7], [18], [41],

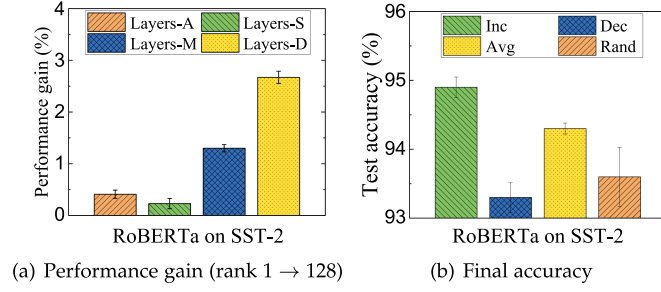


Fig. 5. The impact of LoRA rank distribution.

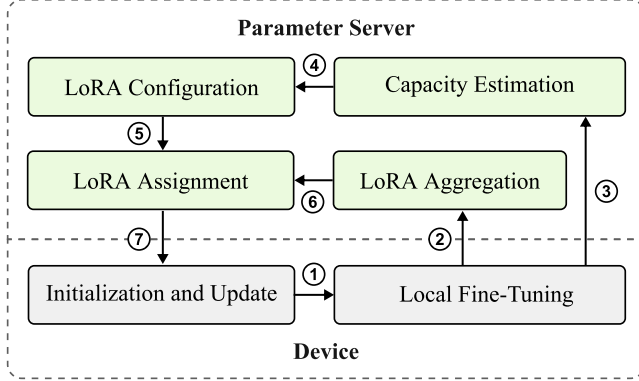


Fig. 6. The proposed LEGEND framework.

[44]. Shallow LoRA layers with smaller ranks leverage the pre-trained model's inherent feature extraction capability, preventing overfitting while maintaining model performance.

To demonstrate the impact of rank distribution, we conduct two sets of experiments for fine-tuning RoBERTa on SST-2. First, we separately apply the different ranks, i.e., $\{1, 2, 4, 8, 16, 32, 64, 128\}$, for Layers-A, Layers-S, Layers-M, and Layers-D (refer to Section II-B) and record their performance gains for comparison. Second, we employ four different rank distributions with a total budget of 96 across 12 transformer layers (from input to output): Inc [4, 4, 5, 6, 7, 7, 8, 9, 10, 11, 12, 13], Dec [13, 12, 11, 10, 9, 8, 7, 7, 6, 5, 4, 4], Avg [8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8], and Rand (randomly allocated). The findings illustrated in Fig. 5 highlight that:

1) Deeper layers are more sensitive to the rank of LoRA. Specifically, applying larger ranks for LoRA layers close to the output helps to achieve better model performance. For instance, as shown in Fig. 5(a), the final performance gains of Layers-A, Layers-S, Layers-M, and Layers-D are separately 0.41%, 0.23%, 2.67%, and 1.3%. This indicates that deeper layers are more parameter-sensitive and contribute more to downstream task performance during fine-tuning.

2) The gradually increasing rank distribution achieves higher model accuracy. For example, as illustrated in Fig. 5(b), Inc distribution achieves the best accuracy (94.9%) among all variants, outperforming others by up to 1.6%. This is because Inc strategically allocates larger ranks to critical deeper layers, greatly enhancing fine-tuning performance. This confirms the advantage of prioritizing deeper layers in rank budget allocation under resource-constrained settings.

Based on the results, it is effective and necessary to allocate larger ranks to deeper layers. However, due to the limited computing and communication resources, allocating larger ranks to deeper layers constrains the applicable LoRA depth, whereas smaller ranks tend to slow down the convergence process. Therefore, in this paper, LEGEND simultaneously determines the appropriate LoRA depth with reasonable rank distribution for different devices so as to balance the trade-off between resource consumption and fine-tuning performance.

III. SYSTEM OVERVIEW

As illustrated in Fig. 6, LEGEND consists of two key components with a total of six main modules, i.e., four modules on the PS and two modules on each device. The details of each module are as follows:

Initialization and Update: The PS distributes different LoRA layers for heterogeneous devices to adapt their capabilities. Hence, the devices need to initialize and update the local model (①) for local fine-tuning in each round.

Local Fine-Tuning: Based on the initialized local model, the devices perform local fine-tuning and record the fine-tuning status information (e.g., computing time and communication time). After that, the devices send the updated LoRA layers (②) and the status information (③) to the PS.

Capacity Estimation: To make effective LoRA configurations, the PS estimates the capabilities of each device (④) by calculating the moving average of the historical status information of the devices.

LoRA Configuration: In this module, the PS simultaneously determines the appropriate LoRA depth with rank distribution, i.e., LoRA configurations (⑤), for the devices.

LoRA Aggregation: The PS performs adaptive weighted aggregation for the collected LoRA layers with different LoRA depths from all devices to obtain the aggregated global LoRA layers (⑥).

LoRA Assignment: Due to the diverse LoRA configurations of devices, the PS needs to assign the specific LoRA layers (⑦) to each device based on the aggregated model.

IV. SYSTEM DESIGN

A. Initialization and Update

In each round, the device receives a set of LoRA layers from the PS to initialize and update the local model for local fine-tuning. These LoRA layers are tailored to the device's capacity (e.g., computing and communication capacity), including information on which transformer layers to fine-tune and the ranks of those LoRA layers. Specifically, device i receives the LoRA layers $\tilde{\theta}_i^h = \{\tilde{\theta}_{i,l}^h | l \in [L - k_i^h, L - 1]\}$ to initialize and update k_i^h transformer layers close to the output, where L denotes the number of transformer layers in the pre-trained model $\bar{\theta}$. The LoRA layers $\tilde{\theta}_{i,l}^h \in \tilde{\theta}_i^h$ includes coupled LoRA matrices for all linear layers in transformer layer l . The coupled LoRA matrices include two components, i.e., the project-down matrix $\mathcal{B} \in \mathbb{R}^{m \times r}$ and the project-up matrix $\mathcal{A} \in \mathbb{R}^{r \times q}$, where

r denotes the rank of the LoRA matrix and is much smaller than both m and q . Without loss of generality, for arbitrary coupled LoRA matrices $\mathcal{B}_{i,l}^h \in \tilde{\theta}_{i,l}^h$ and $\mathcal{A}_{i,l}^h \in \tilde{\theta}_{i,l}^h$, the device injects the matrices into the linear layer $y = M_l \cdot x$ with pre-trained weight matrix M_l in transformer layer l as $\{(\mathcal{B}_{i,l}^h, \mathcal{A}_{i,l}^h, M_l) | l \in \mathcal{L}_i^h\}$. Then, the forward propagation of the linear layer with initialized parameters $\{\mathcal{B}_{i,l}^h, \mathcal{A}_{i,l}^h, M_l\}$ in l th transformer layer on device i in round h can be expressed as:

$$y = M_l \cdot x + \mathcal{B}_{i,l}^h \cdot \mathcal{A}_{i,l}^h \cdot x \quad (5)$$

where x is the given input tensor and y is the respective output tensor of the initialized linear layer. After that, the device initializes and updates the local model $\theta_i^h = \{\tilde{\theta}_i^h, \bar{\theta}\}$ for local fine-tuning in round h .

B. Local Fine-Tuning

After local model initialization, device i fine-tunes the model on local dataset $\mathbb{D}_i$. During the process of local fine-tuning in round h , device i is associated with the local loss function $f_i(\theta_i^h)$, where $\theta_i^h = \{\tilde{\theta}_i^h, \bar{\theta}\}$ is the local model. The loss of device i on the local dataset $\mathbb{D}_i$ in round h can be expressed as follows:

$$f_i(\theta_i^h) = \frac{1}{|\mathbb{D}_i|} \sum_{\xi \in \mathbb{D}_i} F_i(\theta_i^h; \xi_i) \quad (6)$$

where ξ_i is a batch of data samples in $\mathbb{D}_i$, and $F_i(\theta_i^h; \xi_i)$ is the local loss over ξ_i . In general, the devices leverage stochastic gradient descent, e.g., AdamW [34], to iteratively update the LoRA layers based on the computed gradients over each batch of data samples in $\mathbb{D}_i$ [7], [11]. Specifically, for a batch of local data samples ξ_i on device i , the process of updating the LoRA layers $\tilde{\theta}_i^h$ on device i at local step t in round h is expressed as:

$$\tilde{\theta}_i^{h,t} = \tilde{\theta}_i^{h,t-1} - \eta \cdot \nabla f_i(\tilde{\theta}_i^{h,t-1}) \quad (7)$$

where η is the learning rate and $\nabla f_i(\tilde{\theta}_i^{h,t-1})$ is the gradient of the loss for LoRA layers $\tilde{\theta}_i^{h,t-1}$. When completing local fine-tuning, the devices send the updated local LoRA layers to the PS for aggregation. Simultaneously, the devices upload the relevant computing and communication information of the current round to PS for device resource estimation.

C. Capacity Estimation

The estimation of device capabilities (e.g., time-varying computing and communication capabilities) is essential for LEGEND to determine reasonable LoRA configurations for heterogeneous devices. For arbitrary device i in round h , LEGEND utilizes μ_i^h to denote the time required for updating all LoRA layers in a transformer layer during backpropagation, which can be recorded by the devices directly during the process of local fine-tuning, to indicate the computing capability. Besides, since the upload bandwidth is usually much smaller than the download bandwidth in typical WANs [6], [45] and the size of LoRA layers is typically less than 1% of the original model size [20]. LEGEND employs the uploading time β_i^h of transmitting a

LoRA layer with unit rank from the device i in round h to the PS to indicate the communication capability.

In round h , PS collects recent computing time $\hat{\mu}_i^h$ and uploading time $\hat{\beta}_i^h$ from device i and maintains the historical status. Then, we use the moving average with the historical status of devices to estimate the capacities of the devices [46]. Accordingly, the PS estimates the computing time μ_i^h and the uploading time β_i^h for device i in round h by calculating the moving average with $\rho \in [0, 1]$ (for example, $\rho = 0.8$ in our experiments) as:

$$\mu_i^h = \rho \cdot \mu_i^{h-1} + (1 - \rho) \cdot \hat{\mu}_i^h, \forall i \in [1, n], \forall h \in [1, H] \quad (8)$$

$$\beta_i^h = \rho \cdot \beta_i^{h-1} + (1 - \rho) \cdot \hat{\beta}_i^h, \forall i \in [1, n], \forall h \in [1, H] \quad (9)$$

The primary focus of this work is not on improving status estimation techniques and advanced methods [47], [48] can be easily integrated into LEGEND.

D. LoRA Configuration

LEGEND addresses the challenges of resource constraints and system heterogeneity by determining appropriate LoRA configurations, i.e., LoRA depth and rank distribution, for heterogeneous devices to promote fine-tuning efficiency. Based on the observation in Sections II-C and II-D, LEGEND adaptively assigns LoRA depth for device i in round h with gradually increasing rank distribution. The LoRA configuration of device i in round h can be denoted as the rank distribution $R_i^h = \{r_{i,l} | l \in [L - k_i^h, L - 1]\}$ that includes the LoRA depth k_i^h , where $r_{i,l}$ is the rank of the LoRA layers in l th transformer layer. For simplicity, we use $\mathcal{L}_i^h = [L - k_i^h, L - 1]$ to represent the index of the deep k_i^h transformer layers. Assuming that the total rank of all L transformer layers is ψ , the constraints of rank distribution for device i in round h are as:

$$r_{i,l-1} \leq r_{i,l} \quad (10)$$

$$\sum_{l \in \mathcal{L}_i^h} r_{i,l} \leq \psi \quad (11)$$

In round h , based on the estimation of computing and communication capacities, the completion time (including computing and communication time) on device i is expressed as:

$$t_i^h = \hat{t}_i + k_i^h \cdot \mu_i^h + \sum_{l \in \mathcal{L}_i^h} r_{i,l} \cdot \beta_i^h \quad (12)$$

where $\hat{t}_i$ represents the computing time of forward propagation in one round of local fine-tuning, $k_i^h \cdot \mu_i^h$ represents the total time for backpropagation during the process of local fine-tuning, and $\sum_{l \in \mathcal{L}_i^h} r_{i,l} \cdot \beta_i^h$ denotes the uploading time. Additionally, the waiting time for device i can be represented as $t^h - t_i^h$, where $t^h = \max\{t_i^h | i \in [1, n]\}$ denotes the completion time of the slowest device in round h . Then, the average waiting time of all devices in round h can be formulated as:

$$\mathcal{W}^h = \frac{1}{n} \sum_{i=1}^n (t^h - t_i^h) \quad (13)$$

Algorithm 1: LoRA Configuration Determination.

Input: Total rank budget ψ ; the completion time of the devices t_i^h ; parameter λ .

Output: LoRA configuration

$$R_i^h = \{r_{i,l} | l \in [L - k_i^h, L - 1]\}.$$

1: **function** LORACONFIGURATION():

2: Calculate the gap between the maximum and minimum LoRA depth in round h

$$k_i^h \leftarrow \lceil L \cdot \frac{t^h - t_{i,\min}^h}{t^h} \rceil;$$

3: Determine the LoRA depth for device i by

$$k_i^h \leftarrow \lceil k_i^h \cdot \frac{t^h - t_i^h}{t^h} \rceil;$$

4: Get global rank distribution $R = \{r_l | l \in [0, L - 1]\}$ using an arithmetic sequence with a common difference of λ , where $r_l = r_{l-1} + \lambda$.

5: Adjust LoRA depth k_i^h to ensure the configuration meets device-specific computing and communication constraints by (14) and (15).

6: Generate LoRA configuration

$$R_i^h = \{r_{i,l}^h | l \in [L - k_i^h, L - 1]\} \text{ for device } i, \text{ where } r_{i,l}^h = r_l \in R.$$

7: **end Function**

We define the computing resource consumption of the LoRA layers in a transformer layer with unit rank during local fine-tuning as c . In addition, we use $\hat{c}$ to denote the computing resource consumption for the pre-trained model $\bar{\theta}$ during the forward propagation, which is a constant. Then, assuming that the total computing resource budgets of device i in round h is C_i^h , the computing resource constraints can be expressed as follows:

$$\hat{c} + \sum_{l \in \mathcal{L}_i^h} r_{i,l} \cdot c \leq C_i^h \quad (14)$$

Similarly, we use b to denote the communication consumption of LoRA layers in a transformer layer with unit rank. Let B_i^h represent the total communication resource budget for device i in round h . Then, the communication resource constraints can then be formulated as:

$$\sum_{l \in \mathcal{L}_i^h} r_{i,l} \cdot b \leq B_i^h \quad (15)$$

Given a specific task in the FedFT system, LEGEND determines an appropriate LoRA configuration R_i^h for device i according to the estimation of the device's resource in round h to minimize the overall fine-tuning time $\sum_{h=1}^H t^h$. Thus, we can formulate the problem as follows:

$$\min \sum_{h=1}^H t^h$$

$$\text{s.t.} \begin{cases} r_{i,l-1} \leq r_{i,l}, & \forall l \in \mathcal{L}_i^h \\ \sum_{l \in \mathcal{L}_i^h} r_{i,l} \leq \psi, & \forall i \in [1, n] \\ t_i^h = t_i + k_i^h \cdot \mu_i^h + \sum_{l \in \mathcal{L}_i^h} r_{i,l} \cdot \beta_i^h, & \forall i \in [1, n] \\ \mathcal{W}^h = \frac{1}{n} \sum_{i=1}^n (t^h - t_i^h) \leq \epsilon, & \forall h \in [1, H] \\ \hat{c} + \sum_{l \in \mathcal{L}_i^h} r_{i,l} \cdot c \leq C_i^h, & \forall i \in [1, n] \\ \sum_{l \in \mathcal{L}_i^h} r_{i,l} \cdot b \leq B_i^h, & \forall i \in [1, n] \end{cases} \quad (16)$$

The first set of inequalities denotes the constraints of rank distribution. The second set of inequalities represents that the rank of all LoRA layers cannot exceed the given budget ψ . The third set of equations represents the total time of local fine-tuning and uploading on device i in round h . The fourth set of inequalities represents the average waiting time of all devices cannot exceed the predefined threshold $\epsilon > 0$. The last two sets of inequalities express that the accumulated computing and communication resources cost cannot exceed the computing and communication resource budgets of device i in round h , respectively.

To solve the problem in (16), we propose a greedy-based LoRA configuration determination algorithm, termed LCD, as shown in Algorithm 1. First, the PS calculates the gap k_i^h between the maximum and minimum LoRA depth (Line 2) according to the device status information. Then, the completion time gap k_i^h between device i and the slowest is used to generate the appropriate LoRA depth to adapt heterogeneous devices (Line 3). For example, the most powerful device is assigned with maximum LoRA depth L , the weakest device is assigned with LoRA depth of $L - k_i^h$, while other devices are assigned values within the gap $[L - k_i^h, L]$ based on their capabilities. Second, the PS gets the reasonable rank distribution $R = \{r_l | l \in [0, L - 1]\}$ for all transformer layers using an arithmetic sequence with a common difference of λ in round h (line 4). We use the parameter λ to guide the generation of the rank distribution, where λ indicates that the rank of LoRA layers added to the transformer layer l is λ greater than that of the adjacent transformer layer $l - 1$, i.e., $r_l = r_{l-1} + \lambda$. For instance, $\lambda = 1$ in our experiments by default. Then, we greedily adjust the LoRA depth to ensure the configuration meets the device's capacity (Line 5). Finally, based on global rank distribution R and LoRA depth k_i^h , the PS generates the LoRA configuration $R_i^h = \{r_{i,l}^h | l \in [L - k_i^h, L - 1]\}$ for device i in round h (Line 6).

E. LoRA Aggregation

After receiving the updated LoRA layers from all devices, the PS performs global aggregation. Since the LoRA depth varies across devices while the rank of each LoRA layer remains consistent across all devices, the PS performs adaptive layer-wise aggregation on the collected LoRA layers, i.e., aggregating each layer based on the number of devices contributing to that layer. We use $\tilde{\theta}_l^{h+1}$ to represent the global LoRA layers in l th transformer layer obtained by aggregating the respective LoRA layers from n_l devices. Formally, the adaptive layer-wise aggregation of LoRA layers in transformer layer l can be expressed

as follows:

$$\tilde{\theta}_l^{h+1} = \frac{1}{n_l} \sum_{i=1}^{n_l} \tilde{\theta}_{i,l}^h \quad (17)$$

After aggregation, PS obtains a set of global LoRA layers $\tilde{\theta}^{h+1} = \{\tilde{\theta}_l^{h+1} | l \in [0, L-1]\}$, which will be used for assigning different LoRA layers for each device.

F. LoRA Assignment

According to the obtained LoRA configuration, the PS assigns specific LoRA layers for each device. Specifically, based on the LoRA configuration $R_i^h = \{r_{i,l} | l \in \mathcal{L}_i^h\}$ and the aggregated set of global LoRA layers that can be expressed as:

$$\tilde{\theta}^h = \{\tilde{\theta}_l^h | l \in [0, L-1]\} \quad (18)$$

LEGEND generates LoRA layers $\tilde{\theta}_i^h$ for device i by selecting the LoRA layers from global LoRA layers $\tilde{\theta}^h$ as follows:

$$\tilde{\theta}_i^h = \{\tilde{\theta}_{i,l}^h | l \in \mathcal{L}_i^h\} \quad (19)$$

where $\tilde{\theta}_{i,l}^h = \tilde{\theta}_l^h \in \tilde{\theta}^h$. Since the LoRA layers are all continuous layers at deep position, there is no need to send a separate LoRA configuration for local initialization and update. Thus, after obtaining the LoRA layers, the PS immediately distributes the LoRA layers to the respective devices.

G. Computational Complexity Analysis

In this section, we present the overall computational complexity of our proposed method and compare it with the typical FedLoRA. For clarity, we denote the computational complexity of forward propagation of a batch of data with max length s in a pre-trained transformer layer as $\bar{U}_f$, and that in a LoRA-enhanced transformer layer with unit rank as $\tilde{U}_f$. Similarly, $\tilde{U}_b$ represents the computational complexity of backward propagation for unit-rank LoRA layers. As a reference, the total computational complexity of a standard FedLoRA system is given by $\mathcal{O}(H \cdot n \cdot T \cdot L \cdot r \cdot (\tilde{U}_f + \tilde{U}_b))$, where n is the number of devices, H is the total number of aggregation rounds, T is the number of local update steps per round, L is the number of transformer layers, and r is the fixed LoRA rank applied uniformly to all layers and devices. In contrast, our method dynamically assigns each device an adaptive fine-tuning depth with a reasonable rank distribution. The resulting computational complexity is $\mathcal{O}(T \cdot \sum_{h=1}^H \sum_{i=1}^n ((L - k_i^h) \cdot \bar{U}_f + \sum_{l=L-k_i^h}^{L-1} r_{i,l}^h \cdot (\tilde{U}_f + \tilde{U}_b)))$, where k_i^h denotes the number of layers fine-tuned by device i in round h , and $r_{i,l}^h$ is the rank assigned to the l th transformer layer of device i in that round. In addition, we only introduce two minimal computational costs: $\mathcal{O}(n \cdot H)$ for computing the device-specific capacity estimation metrics (e.g., μ_i^h , β_i^h) and $\mathcal{O}(n \cdot H \cdot L)$ for generating layer-wise LoRA configurations on the server side. Thus, the total computational complexity of our method can be expressed as $\mathcal{O}(T \cdot \sum_{h=1}^H \sum_{i=1}^n ((L - k_i^h) \cdot \bar{U}_f + \sum_{l=L-k_i^h}^{L-1} r_{i,l}^h \cdot (\tilde{U}_f + \tilde{U}_b))) + \mathcal{O}(n \cdot H \cdot (L+1))$. It is worth noting that during

model fine-tuning, backpropagation generally incurs approximately twice the computational cost of forward propagation, constituting the dominant component of computational complexity [49], [50], [51]. Thus, compared to the uniform and full-layer LoRA tuning in FedLoRA, our approach significantly reduces the computational overhead by assigning adaptive tuning depth with reasonable rank distribution tailored to device capabilities.

V. THEORETICAL ANALYSIS

In this section, we theoretically analyze the rationale behind the rank distribution and provide a convergence analysis of the proposed LEGEND framework.

A. Theoretical Insight

Theorem 5.1: Under standard regularity conditions, during the fine-tuning of a pre-trained model, the expected squared norm of the gradients with respect to the parameters tends to be larger for layers closer to the output layer compared to layers closer to the input layer. That is, there is a tendency for:

$$\begin{aligned} \mathbb{E}_{\xi \sim \mathbb{D}} [\|\nabla_{\theta_L} f(\xi, \theta)\|_2^2] &\gtrsim \mathbb{E}_{\xi \sim \mathbb{D}} [\|\nabla_{\theta_{L-1}} f(\xi, \theta)\|_2^2] \\ &\gtrsim \cdots \gtrsim \mathbb{E}_{\xi \sim \mathbb{D}} [\|\nabla_{\theta_1} f(\xi, \theta)\|_2^2] \end{aligned} \quad (20)$$

Moreover, the expected norm of the gradients decays approximately exponentially as we move from deeper layers towards shallower layers.

Proof: The gradient of the loss function f with respect to the parameters θ_l of layer l can be computed using the chain rule for backpropagation:

$$\nabla_{\theta_l} f = \frac{\partial f}{\partial f_L} \cdot \frac{\partial f_L}{\partial f_{L-1}} \cdots \frac{\partial f_{l+1}}{\partial f_l} \cdot \frac{\partial f_l}{\partial \theta_l}$$

Let $J_k = \frac{\partial f_{k+1}}{\partial f_k}$ denote the Jacobian matrix of the transformation at layer k . Using the property of matrix norms $\|AB\| \leq \|A\| \|B\|$, we can write:

$$\|\nabla_{\theta_l} f\|_2 \leq \left\| \frac{\partial f}{\partial f_L} \right\|_2 \cdot \left(\prod_{k=l}^{L-1} \|J_k\|_{op} \right) \cdot \left\| \frac{\partial f_l}{\partial \theta_l} \right\|_2$$

where $\|\cdot\|_{op}$ denotes the operator norm. It is often observed or assumed that in pre-trained networks, these Jacobian norms are bounded [52], i.e., $\|J_k\|_{op} \leq \gamma_k$. If, particularly for layers further from the task-specific head, these γ_k are effectively less than 1, the product term $\prod_{k=l}^{L-1} \|J_k\|_{op}$ can attenuate the gradient signal. Specifically, if we assume $\|J_k\|_{op} \leq \gamma_{decay} < 1$ for many k , then

$$\|\nabla_{\theta_l} f\|_2 \leq \left\| \frac{\partial f}{\partial f_L} \right\|_2 \cdot \gamma_{decay}^{L-l} \cdot \left\| \frac{\partial f_l}{\partial \theta_l} \right\|_2$$

This inequality shows that the upper bound on the gradient norm decays exponentially as l decreases, and gradients in layers closer to the output (larger l) are likely to have larger magnitudes. $\square$

Theorem 5.2: Consider an optimal weight update $\Delta W_l^* \in \mathbb{R}^{D_{out,l} \times D_{in,l}}$ for weight matrix $W_l^* \in \mathbb{R}^{D_{out,l} \times D_{in,l}}$, where

$D_{out,l}$ and $D_{in,l}$ denotes the input and output dimensions of layer l , respectively. If we use a LoRA modules $B_l A_l$, where $B_l \in \mathbb{R}^{D_{out,l} \times r_l}$ and $A_l \in \mathbb{R}^{r_l \times D_{in,l}}$, such that $B_l A_l$ is a matrix of rank at most r_l . The Eckart-Young-Mirsky theorem [53] states that the best possible approximation of ΔW_l^* by any matrix of rank r_l is achieved by its truncated SVD. The squared Frobenius norm of the error for this best rank- r_l approximation is:

$$\inf_{\text{rank}(\widehat{W}) \leq r_l} \left\| \Delta W_l^* - \widehat{W} \right\|_F^2 = \sum_{j=r_l+1}^{\min(D_{out,l}, D_{in,l})} \sigma_j^2(\Delta W_l^*)$$

where $\sigma_j(\Delta W_l^*)$ denotes the j th singular value of ΔW_l^* (sorted in non-increasing order) and $\|\cdot\|_F$ represent the Frobenius norm. Thus, increasing the LoRA rank r_l reduces the minimum achievable approximation error for ΔW_l^* , providing greater capacity for the LoRA adapter to capture the optimal update.

Proof: The LoRA update for weight matrix W_l is $\Delta W_l = B_l A_l$. By construction, the rank of ΔW_l is at most r_l . We are interested in how well this rank- r_l update can approximate an optimal, potentially full-rank, update ΔW_l^* . The Eckart-Young-Mirsky theorem states that for any matrix ΔW_l^* , its best rank- k approximation in the Frobenius norm (and spectral norm) is obtained by its truncated Singular Value Decomposition (SVD). Let $\Delta W_l^* = U \Sigma V^T$ be the SVD of ΔW_l^* , where $\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p)$ with $p = \min(D_{out,l}, D_{in,l})$. The best rank- r_l approximation, denoted $\Delta W_l^{*(r_l)}$, is $U_{r_l} \Sigma_{r_l} V_{r_l}^T$, where U_{r_l} and V_{r_l} are the first r_l columns of U and V respectively, and $\Sigma_{r_l} = \text{diag}(\sigma_1, \dots, \sigma_{r_l})$. The minimum squared Frobenius error achievable by any rank- r_l matrix $\widehat{W}$ in approximating ΔW_l^* is thus:

$$\begin{aligned} \left\| \Delta W_l^* - \Delta W_l^{*(r_l)} \right\|_F^2 &= \left\| U \Sigma V^T - U_{r_l} \Sigma_{r_l} V_{r_l}^T \right\|_F^2 \\ &= \left\| U (\Sigma - \Sigma^{(r_l)}) V^T \right\|_F^2, \end{aligned}$$

where $\Sigma^{(r_l)}$ is Σ with singular values $\sigma_j = 0$ for $j > r_l$. Since U and V are orthogonal, the Frobenius norm is preserved:

$$\left\| \Sigma - \Sigma^{(r_l)} \right\|_F^2 = \sum_{j=r_l+1}^{\min(D_{out,l}, D_{in,l})} \sigma_j^2(\Delta W_l^*)$$

While the LoRA update $B_l A_l$ is learned through gradient descent and may not perfectly achieve this SVD-based optimum, this theorem establishes that the potential for approximation improves as r_l increases because the sum of squared discarded singular values (the minimum error) decreases. A larger LoRA rank r_l provides more capacity to the model $B_l A_l$ to better approximate ΔW_l^* . $\square$

Remark 5.3: Theorems 5.1 and 5.2 together suggest a principled approach for allocating LoRA ranks across layers. From Theorem 5.1, we know that layers closer to the output tend to have larger gradient magnitudes, suggesting their updates ΔW_l^* would contain more significant information. Layers with larger gradient magnitudes (closer to output) likely have larger leading singular values $\sigma_{1,l}$. To minimize the total approximation error $\sum_{l=0}^{L-1} (\sum_{j=r_l+1}^{\min(D_{out,l}, D_{in,l})} \sigma_j^2(\Delta W_l^*))$ under a fixed total rank

budget $\sum_{l=0}^{L-1} r_l = \psi$, layers with slower singular value decay (smaller α_l) and/or larger leading singular values should receive higher rank allocations. This implies an efficient allocation strategy would assign higher ranks to layers closer to the output layer, which aligns with the empirical observations in Section II-D.

B. Convergence Analysis

In this section, we provide a convergence analysis for the proposed framework. First, we make the following commonly used assumptions [18], [54], [55].

Assumption 5.1 (Lipschitz Continuous Gradient): The loss function f_i is with $\mathcal{L}_{lip}$ -Lipschitzian gradients, $\forall \theta_1, \theta_2$, i.e.,

$$\|\nabla f_i(\theta_1) - \nabla f_i(\theta_2)\|_2 \leq \mathcal{L}_{lip} \|\theta_1 - \theta_2\|_2, \forall i \quad (21)$$

where $\|\cdot\|_2$ is the L_2 norm of vector.

Assumption 5.2 (Unbiased estimation): For each device i , we assume that the expectation of the stochastic gradient of F on each data sample is equal to $f_i(\theta)$, i.e.,

$$\mathbb{E}_{\xi \sim \mathbb{D}_i} [\nabla f_i(\theta; \xi)] = \nabla f_i(\theta) \quad (22)$$

And further, for all devices, we assume that,

$$\mathbb{E}_{i \in [n]} \nabla f_i(\theta) = \nabla f(\theta). \quad (23)$$

Assumption 5.3 (Bounded Gradient Variance): We assume the stochastic gradient noise within each device is bounded:

$$\mathbb{E}_{\xi \sim \mathbb{D}_i} \|\nabla f_i(\theta; \xi) - \nabla f_i(\theta)\|^2 \leq \varrho^2, \quad \forall i \in [n], \forall \theta. \quad (24)$$

Moreover, the variance of local gradients with respect to the global gradient is bounded as

$$\mathbb{E}_{i \in [n]} \|\nabla f_i(\theta) - \nabla f(\theta)\|^2 \leq \varsigma^2, \quad \forall \theta, \quad (25)$$

where $\varsigma^2 = \varsigma_d^2 + \varsigma_c^2$ reflects both data heterogeneity ς_d^2 and approximation error induced by depth-truncated LoRA ς_c^2 . The former arises under standard non-independent and identically distributed assumptions, while the latter stems from structured low-rank adaptation and layer truncation. The approximation error introduced by the LoRA and layer truncation is assumed to be bounded, with upper bound ς_c^2 , justified theoretically in Theorem 5.2 and controlled under the Lipschitz condition and limited model depth.

Theorem 5.4 (Convergence of LEGEND): Let Assumptions 5.1–5.3 hold. Then, after H communication rounds, LEGEND outputs a model $\bar{\theta}$ such that:

$$\frac{1}{H} \sum_{h=1}^H \mathbb{E} \|\nabla f(\theta^h)\|^2 \leq \frac{4(f(\theta^1) - f(\theta^*))}{\eta H} + 8(\varrho^2 + \varsigma^2) \quad (26)$$

where θ^* denotes the optimal model parameters and $\eta \leq \frac{1}{2\mathcal{L}_{lip}}$ is the learning rate.

Proof: Let θ^h be the global model at communication round h , and define $g^h = \frac{1}{n} \sum_{i=1}^n g_i^h$ as the aggregated update direction from n devices. Under standard SGD updates, we have $\theta^{h+1} = \theta^h - \eta g^h$. By Assumption 5.1, we get:

$$f(\theta^{h+1}) \leq f(\theta^h) + \langle \nabla f(\theta^h), \theta^{h+1} - \theta^h \rangle$$

$$+ \frac{\mathcal{L}_{lip}}{2} \|\theta^{h+1} - \theta^h\|^2.$$

Then, we substitute the update rule:

$$f(\theta^{h+1}) \leq f(\theta^h) - \eta \langle \nabla f(\theta^h), g^h \rangle + \frac{\mathcal{L}_{lip}\eta^2}{2} \|g^h\|^2.$$

Taking expectation and applying the inequality $-\langle a, b \rangle \leq \frac{1}{2}\|a\|^2 + \frac{1}{2}\|b\|^2$, we obtain:

$$\begin{aligned} \mathbb{E}[f(\theta^{h+1})] &\leq \mathbb{E}[f(\theta^h)] - \frac{\eta}{2} \mathbb{E} \|\nabla f(\theta^h)\|^2 \\ &\quad + \frac{\eta}{2} \mathbb{E} \|g^h - \nabla f(\theta^h)\|^2 + \frac{\mathcal{L}_{lip}\eta^2}{2} \mathbb{E} \|g^h\|^2. \end{aligned} \quad (27)$$

Then, according to Assumption 5.3, we can bound the variance terms. For the first term:

$$\begin{aligned} \mathbb{E} \|g^h - \nabla f(\theta^h)\|^2 &= \mathbb{E} \left\| \frac{1}{n} \sum_{i=1}^n g_i^h - \nabla f(\theta^h) \right\|^2 \\ &= \mathbb{E} \left\| \frac{1}{n} \sum_{i=1}^n (g_i^h - \nabla f_i(\theta^h)) \right. \\ &\quad \left. + \frac{1}{n} \sum_{i=1}^n (\nabla f_i(\theta^h) - \nabla f(\theta^h)) \right\|^2 \\ &\leq 2\mathbb{E} \left\| \frac{1}{n} \sum_{i=1}^n (g_i^h - \nabla f_i(\theta^h)) \right\|^2 \\ &\quad + 2\mathbb{E} \left\| \frac{1}{n} \sum_{i=1}^n (\nabla f_i(\theta^h) - \nabla f(\theta^h)) \right\|^2 \\ &\leq 2\varrho^2 + 2\varsigma^2. \end{aligned} \quad (28)$$

Similarly, the squared norm of g^h is bounded by:

$$\begin{aligned} \mathbb{E} \|g^h\|^2 &= \mathbb{E} \|g^h - \nabla f(\theta^h) + \nabla f(\theta^h)\|^2 \\ &\leq 2\mathbb{E} \|g^h - \nabla f(\theta^h)\|^2 + 2\mathbb{E} \|\nabla f(\theta^h)\|^2 \\ &\leq 2(2\varrho^2 + 2\varsigma^2) + 2\mathbb{E} \|\nabla f(\theta^h)\|^2 \\ &= 4\varrho^2 + 4\varsigma^2 + 2\mathbb{E} \|\nabla f(\theta^h)\|^2. \end{aligned} \quad (29)$$

Plugging into the earlier smoothness inequality:

$$\begin{aligned} \mathbb{E}[f(\theta^{h+1})] &\leq \mathbb{E}[f(\theta^h)] - \frac{\eta}{2} \mathbb{E} \|\nabla f(\theta^h)\|^2 + \eta(\varrho^2 + \varsigma^2) \\ &\quad + \mathcal{L}_{lip}\eta^2 \left(\mathbb{E} \|\nabla f(\theta^h)\|^2 + 2\varrho^2 + 2\varsigma^2 \right) \\ &= \mathbb{E}[f(\theta^h)] - \left(\frac{\eta}{2} - \mathcal{L}_{lip}\eta^2 \right) \mathbb{E} \|\nabla f(\theta^h)\|^2 \\ &\quad + \eta(\varrho^2 + \varsigma^2) (1 + 2\mathcal{L}_{lip}\eta). \end{aligned} \quad (30)$$

For convergence, we require $\frac{\eta}{2} - \mathcal{L}_{lip}\eta^2 > 0$, which holds when $\eta < \frac{1}{2\mathcal{L}_{lip}}$. Assuming $\eta \leq \frac{1}{2\mathcal{L}_{lip}}$, we have $\frac{\eta}{2} - \mathcal{L}_{lip}\eta^2 \geq 0$

and $(1 + 2\mathcal{L}_{lip}\eta) \leq 2$. This gives us:

$$\mathbb{E}[f(\theta^{h+1})] \leq \mathbb{E}[f(\theta^h)] - \frac{\eta}{4} \mathbb{E} \|\nabla f(\theta^h)\|^2 + 2\eta(\varrho^2 + \varsigma^2). \quad (31)$$

By rearranging terms, we have:

$$\frac{\eta}{4} \mathbb{E} \|\nabla f(\theta^h)\|^2 \leq \mathbb{E}[f(\theta^h)] - \mathbb{E}[f(\theta^{h+1})] + 2\eta(\varrho^2 + \varsigma^2). \quad (32)$$

Then, summing over communication rounds $h = 1$ to H and noting that $\mathbb{E}[f(\theta^{H+1})] \geq f(\theta^*)$, we derive:

$$\begin{aligned} \frac{\eta}{4} \sum_{h=1}^H \mathbb{E} \|\nabla f(\theta^h)\|^2 &\leq \mathbb{E}[f(\theta^1)] - \mathbb{E}[f(\theta^{H+1})] \\ &\quad + 2\eta H(\varrho^2 + \varsigma^2) \\ &\leq f(\theta^1) - f(\theta^*) + 2\eta H(\varrho^2 + \varsigma^2). \end{aligned} \quad (33)$$

Dividing both sides by $\frac{\eta H}{4}$, we have:

$$\frac{1}{H} \sum_{h=1}^H \mathbb{E} \|\nabla f(\theta^h)\|^2 \leq \frac{4(f(\theta^1) - f(\theta^*))}{\eta H} + 8(\varrho^2 + \varsigma^2). \quad (34)$$

The first term $\frac{4(f(\theta^1) - f(\theta^*))}{\eta H}$ decreases as the number of communication rounds H increases, reflecting the convergence of the model parameters towards a stationary point. The second term $8(\varrho^2 + \varsigma^2)$ represents the error floor caused by stochastic gradient noise ϱ^2 and heterogeneity plus LoRA approximation and truncation errors ς^2 . By now, we have proved Theorem 5.4, and our proposed LEGEND framework can achieve convergence during fine-tuning. $\square$

VI. IMPLEMENTATION

We implement our FedFT prototype based on the open-source FedPETuning framework [56], extending its functionality with approximately 2.1 K lines of custom code. The prototype is designed to support heterogeneous computing platforms, with a specific focus on commercial NVIDIA Jetson devices [57], including Jetson TX2, Jetson NX, and Jetson AGX. The software platform is built based on Docker Swarm [58], [59], a distributed software development kit that helps build distributed systems with the ability to monitor the status of each device. For computing, we utilize PyTorch [60] to facilitate the implementation of model fine-tuning on devices, ensuring platform independence while allowing platform-specific backend acceleration. To accelerate the on-device fine-tuning, we utilize NVIDIA-provided development packages² to take full advantage of the underlying hardware capabilities. For communication, we adopt MPI (Message Passing Interface) [61], which includes a collection of sending and receiving functions, e.g., *comm.send(data, dest, tag)/comm.recv(sour, tag)*, to streamline communication between the PS and devices. The prototype provides simple

²<https://forums.developer.nvidia.com/t/pytorch-for-jetson/72048>

TABLE I
TECHNICAL OVERVIEW OF JETSON PLATFORMS

Jetson	AI Performance	GPU Type
TX2	1.33 TFLOPS	256-core Pascal
NX	21 TOPS	384-core Volta
AGX Xavier	22 TOPS	512-core Volta
Jetson	CPU Type	ROM
TX2	Denver 2 and ARM 4	8 GB LPDDR4
NX	6-core Carmel ARM 8	8 GB LPDDR4x
AGX Xavier	8-core Carmel ARM 8	32 GB LPDDR4x

TABLE II
OVERVIEW OF DATASETS FOR EXPERIMENTAL EVALUATION

Dataset	Partition Rules	# Training Samples	# Test Samples
SST-2	non-IID	67,349	1,821
QNLI	non-IID	104,743	5,463
QQP	non-IID	363,846	40,430
MNLI	non-IID	392,702	9,815
GSM-8K	IID	7473	1,319
MMLU	IID	20,000	2,000

APIs to abstract away the problem of federated fine-tuning on heterogeneous computing platforms. For example, in the PS, we use docker API *docker stack deploy* to deploy the fine-tuning process on both PS and the heterogeneous device with the customized container for different devices. The implementation addresses key challenges in supporting heterogeneous devices by developing flexible communication and fine-tuning protocols that can adapt to varying computational resources.

VII. EVALUATION

A. Methodology

Experimental Setup: Extensive experiments are conducted on the implemented prototype system with one PS and 80 devices to evaluate the performance of LEGEND. Specifically, the PS runs on a workstation equipped with an Intel(R) Xeon(R) Platinum 8358P CPU (@ 2.60 GHz with 128 cores), 8 NVIDIA RTX A6000 GPUs (48 GB memory each) and 512 GB RAM. In addition, we specify 80 NVIDIA commercial developer kits, including 30 Jetson TX2 kits, 40 Jetson NX kits, and 10 Jetson AGX kits, as devices to construct a heterogeneous system. The detailed technical specifications of Jetson TX2, NX, and AGX kits are listed in Table I.

Settings of System Heterogeneity: To emulate the heterogeneous computing and communication capabilities among devices, we present the following setups.

(1) *For computing:* By specifying different modes of the Jetson devices (i.e., Jetson TX2, NX, and AGX), our prototype system enables these devices to work with varying computing capabilities. Specifically, Jetson TX2 offers four configurable modes, whereas the Jetson NX and AGX support up to eight modes. For example, the Jetson AGX with the mode 0 (i.e., the highest performance mode of Jetson AGX) achieves fine-tuning by 100× faster than the TX2 with the mode 1 (i.e., the lowest performance mode of Jetson TX2). Besides, to reflect resource

varying over time, the devices are configured to randomly change the mode every 20 rounds.

(2) *For communication:* To replicate the practical network environment, all devices are connected to the PS via Wi-Fi routers in the prototype system. Concretely, the devices are randomly shuffled and divided into four groups, with each group containing 20 devices. Then, these groups are placed at different locations, i.e., 2 m, 8 m, 14 m, and 20 m, away from the Wi-Fi routers. Due to random channel noise and competition among devices, the bandwidth between the PS and devices varies dynamically during the fine-tuning. The bandwidth of devices is measured by iperf3 [62], which fluctuates between 1 Mb/s and 30 Mb/s.

Tasks and Models: We evaluate the performance of LEGEND using three representative models downloaded from Hugging Face [63], i.e., RoBERTa [39], DeBERTa [64], and Llama [15], across the three categories of tasks, including general language understanding [40], massive multitask understanding [65] and mathematical reasoning [66], [67], as shown in Table II.

1) *General Language Understanding* aims to assess the natural language understanding capabilities of the models. We pick four datasets from the General Language Understanding Evaluation (GLUE) datasets [40], encompassing a diverse spectrum of natural language understanding challenges: SST-2 for sentiment analysis, QNLI for question-based natural language inference, QQP for semantic equivalence, and MNLI for multi-genre textual entailment, respectively. Following the *Dirichlet* distribution with $\alpha = 10$, we build the non-independent and identically (non-IID) distributed datasets [7]. We fine-tune a 125M-parameter RoBERTa-base model [39] with 12 transformer layers on SST-2 and QNLI, and a 350M-parameter DeBERTa-Large model [64] with 24 transformer layers on QQP and MNLI, respectively.

2) *Massive Multitask Understanding* aims to test model generalization and knowledge transfer by fine-tuning across multiple related tasks. We utilize the Massive Multitask Language Understanding (MMLU) datasets [65] for the evaluation, which is a comprehensive multiple-choice benchmark spanning 57 tasks across diverse academic domains. By sampling 20,000 training and 2,000 test samples from the *auxiliary_train* subset, we create a dataset that challenges models to demonstrate broad knowledge and nuanced reasoning capabilities. We fine-tune a 7B-parameter Llama2 model (i.e., Llama2-7B) [15], which is composed of 32 transformer layers, on this customized dataset.

3) *Mathematical Reasoning* serves as a critical task for evaluating the models' logical reasoning and computational reasoning capabilities. We employ the Grade School Math (GSM-8 K) dataset [67], which is a widely recognized benchmark designed to evaluate models' elementary mathematical reasoning and problem-solving abilities. It comprises 7,473 training and 1,319 test problems, each requiring multi-step reasoning and arithmetic calculations, reflecting the complexity of grade-school mathematics. This dataset challenges models to demonstrate logical consistency, numerical computation, and interpretive skills essential for advanced mathematical understanding. We fine-tune Llama2-7B on this dataset.

Baselines: To evaluate the effectiveness of LEGEND, we adopt two LoRA-based approaches (i.e., naive FedLoRA [20]

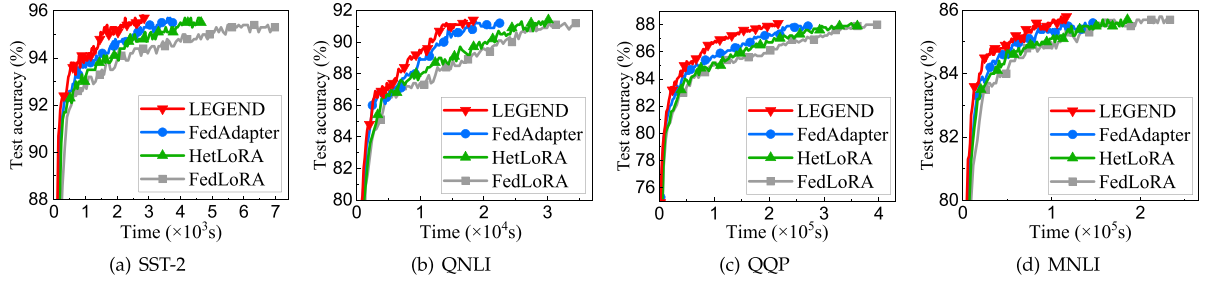


Fig. 7. Fine-tuning process of four approaches on general language understanding tasks.

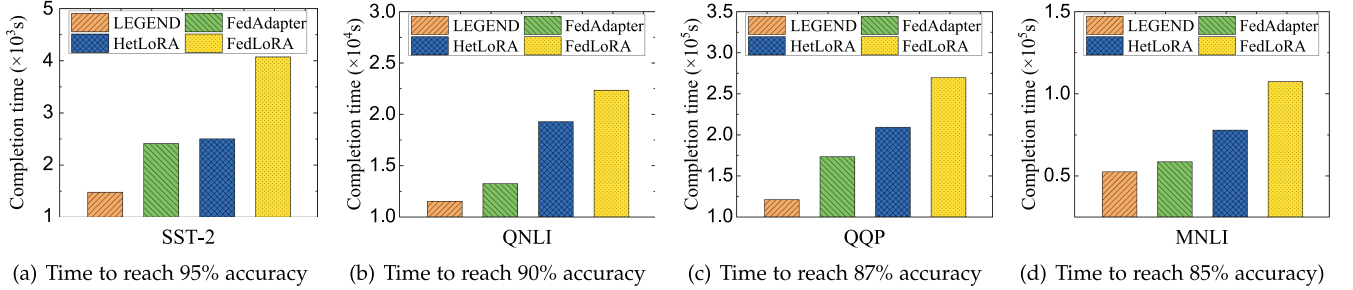


Fig. 8. Completion time of four approaches on general language understanding tasks.

and advanced HetLoRA [27]) and the state-of-the-art FedFT approaches (i.e., FedAdapter [11]) as baselines.

1) *FedLoRA* integrates LoRA [16] into FedFT, where all the devices fine-tune the same local model with the identical rank applied to all transformer layers.

2) *HetLoRA* is an advanced LoRA-based approach for FedFT, which assigns each device with a diverse but appropriate LoRA rank for fine-tuning all transformer layers of its local model so as to deal with system heterogeneity.

3) *FedAdapter* is the state-of-the-art FedFT approach, which introduces Adapters [21] in FedFT and dynamically searches for the optimal Adapter configuration to improve fine-tuning efficiency.

Metrics: The following metrics are adopted to evaluate the performance of LEGEND and the baselines.

1) *Test Accuracy* reflects the accuracy of the models fine-tuned by different approaches on the test datasets, measured by the proportion of correctly predicted data. Specifically, we record the test accuracy of the global model (the model after aggregation at the PS) in each round.

2) *Completion time* represents the total wall-clock time required for fine-tuning a model to achieve a target accuracy. For fair comparisons, we set the target accuracy as the minimum accuracy achieved by the four methods. We record the time of each round, summing it up to obtain the completion time, and also record the average waiting time to reflect the fine-tuning efficiency of different approaches.

3) *Communication Traffic* is recorded by summing up the traffic for transmitting the trainable parameters between the PS and devices during model fine-tuning, which is used to measure the communication efficiency of each approach.

Experimental Parameters: By default, all experiments are carried out on our prototype system and run 100 rounds. Each

device fine-tunes locally for 1 epoch per round using the AdamW optimizer [34], and the learning rate is $5e-4$, which decays according to a cosine scheduler. The batch size is fixed at 4 and the maximum sequence length is set to 512 for all the experiments.

B. Overall Performance

First, we conduct sets of experiments to evaluate the performance of LEGEND and the baselines. The fine-tuning processes and the completion time of the general language understanding task are presented in Figs. 7 and 8, respectively. By the results, LEGEND achieves the fastest convergence rate, outperforming the other approaches by a significant margin on all tasks. By assigning smaller LoRA depths with an optimized rank distribution for resource-constrained devices, LEGEND effectively enhances fine-tuning performance while reducing the time for local fine-tuning. For instance, by Figs. 7(a) and 8(a), LEGEND takes only 1,479 s to achieve 95% accuracy on SST-2, while FedAdapter, HetLoRA, and FedLoRA take 2,412 s, 2,503 s, and 4,074 s, respectively. Compared to FedAdapter, HetLoRA, and FedLoRA, LEGEND provides $1.6\times$, $1.7\times$, and $2.8\times$ speedup, respectively. By Figs. 7(b) and 8(b), LEGEND also outperforms the other baselines in terms of completion time for QNLI. Similarly, Figs. 7(c) and 8(c) show that LEGEND takes 121,156 s to achieve 87% accuracy for QQP, while FedAdapter, HetLoRA, and FedLoRA consume 173,375 s, 209,196 s, and 269,749 s, respectively. LEGEND separately achieves a speedup of $1.4\times$, $1.7\times$, and $2.2\times$, compared with FedAdapter, HetLoRA, and FedLoRA. For MNLI, as shown in Figs. 7(d) and 8(d), LEGEND speeds up the fine-tuning process by about $1.2\times$, $1.5\times$, and $2.1\times$, compared to FedAdapter, HetLoRA, and FedLoRA, respectively. These results demonstrate the superiority of LEGEND in

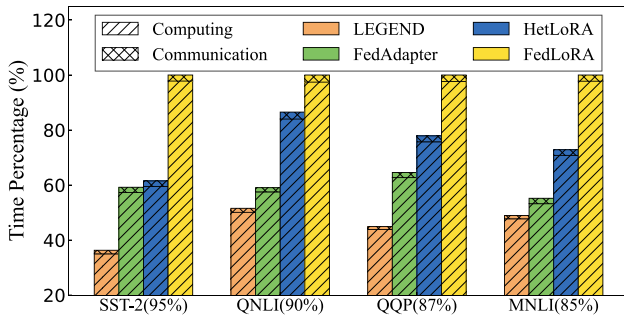


Fig. 9. Time breakdown of four approaches when reaching different target accuracies.

accelerating the fine-tuning process through joint optimization of LoRA depth and rank distribution.

Furthermore, we analyze the impact of speed improvements versus communication cost reduction. We present the time breakdown of all methods during fine-tuning in Fig. 9 to show the sources of speed improvements. Although our method can effectively reduce communication overhead, the speed improvements mainly come from computational cost savings. For example, as shown in Fig. 9, LEGEND achieves the fastest convergence across different datasets, with improvements in computational latency dominating, accounting for over 97% of the overall completion time reduction to reach the target accuracy, while communication overhead improvements account for approximately 1-2.4%. While parameter-efficient fine-tuning methods greatly alleviate communication overhead by transmitting fewer parameters, our results show that the dominant bottleneck in federated fine-tuning stems from the constrained computing capabilities of the devices.

Second, to illustrate the advantage of LEGEND in saving communication resources, we evaluate the communication traffic consumption of these approaches when achieving the target accuracy. According to the results in Fig. 10, LEGEND always causes the least communication traffic among all approaches. For example, as shown in Fig. 10(a), when achieving 95% accuracy, LEGEND consumes 9.9 GB, while FedAdapter, HetLoRA, and FedLoRA consume 14.2 GB, 14.7 GB, and 16.2 GB, respectively. Compared with FedAdapter, HetLoRA, and FedLoRA, LEGEND reduces communication traffic by about 30.0%, 32.1%, and 38.3%, respectively. On one hand, compared to FedLoRA and HetLoRA which add LoRA layers of the same rank to all layers, LEGEND generates adaptive LoRA depth and rank distribution to accommodate heterogeneous devices. On the other hand, unlike FedAdapter which requires multiple device groups to find the optimal adapter structure, LEGEND determines the suitable LoRA configuration based on the device's status, making fine-tuning more efficient and thus reducing communication overhead. Besides, by Fig. 10(b), LEGEND consumes 12.1 GB when achieving 90% accuracy for QNLI, reducing the communication traffic by about 1.95 GB, 5.46 GB, and 5.74 GB compared with FedAdapter, HetLoRA, and FedLoRA, respectively. For QQP, the results in Fig. 10(c) show that LEGEND reduces the total traffic consumption up to 42.3% when achieving accuracy. Moreover, by Fig. 10(d), when achieving 85% accuracy for MNLI, LEGEND consumes

42 GB, while FedAdapter, HetLoRA, and FedLoRA consume about 54 GB, 60 GB, and 69 GB, respectively. The experimental results underscore the advantages of LEGEND in reducing communication costs.

To demonstrate the effectiveness of LEGEND towards system heterogeneity, we illustrate the average waiting time of the four approaches on the four datasets in Fig. 11. LEGEND achieves the shortest waiting time across all datasets by assigning different LoRA depths with reasonable rank distribution to heterogeneous devices. For example, by Fig. 11(a), LEGEND separately improves the average waiting time by about 11.0%, 19.5%, and 40.2%, compared with FedAdapter, HetLoRA, and FedLoRA for SST-2. By Fig. 11(b), the average waiting time of LEGEND is 38.6 s for QNLI, while FedAdapter, HetLoRA, and FedLoRA incur average waiting time of 50.3 s, 57.4 s, and 76.7 s, respectively. LEGEND reduces the average waiting time by about 23.4%, 32.8%, and 49.7% compared to FedAdapter, HetLoRA, and FedLoRA, respectively. Besides, as illustrated by Fig. 11(c), the average waiting time of LEGEND is 352 s for SST-2 while FedAdapter, HetLoRA, and FedLoRA incur an average waiting time of 532 s, 623 s, and 710 s, respectively. Compared with FedAdapter, HetLoRA, and FedLoRA, LEGEND reduces the average waiting time by about 33.8%, 43.5%, and 50.4%, respectively. In addition, by Fig. 11(d), LEGEND also outperforms FedAdapter, HetLoRA, and FedLoRA, reducing the averaging waiting time by 14%, 24%, and 32.7%, respectively. LEGEND achieves the least waiting time across all tasks by assigning appropriate depth to devices, significantly reducing the time for local fine-tuning on resource-constrained devices. In contrast, FedLoRA adds LoRA layers to all transformer layers of the model, which requires the complete backpropagation process to update all LoRA layers and fast devices should be forced to wait for slow ones, leading to prolonged waiting time and poor fine-tuning efficiency. HetLoRA improves FedLoRA by assigning diverse ranks to the devices to alleviate the challenge of system heterogeneity but with limited success, while FedAdapter speeds up the fine-tuning process by selecting the best fine-tuning group from groups with different Adapter configurations in each round. These experimental results demonstrate the superiority of LEGEND in mitigating system heterogeneity.

C. Results on MMLU and GSM-8 K

As a complement to general natural language understanding tasks, we further validated our approach on MMLU and GSM-8 K to explore whether LEGEND handles generation or multi-step reasoning tasks effectively. We present the experimental results for massive multitask understanding and mathematical reasoning tasks in Fig. 12. LEGEND consistently outperforms all other baselines, significantly accelerating the fine-tuning process. Specifically, as shown in Fig. 12(a) and (b), LEGEND achieves a speedup of 1.3 $\times$, 1.5 $\times$, and 2.3 $\times$ compared to FedAdapter, HetLoRA, and FedLoRA, respectively. In Fig. 12(c) and (d), LEGEND reduces the total completion time required to achieve 30% accuracy by approximately 26%, 41%, and 54% when compared to FedAdapter, HetLoRA, and FedLoRA, respectively. The experimental results shown in Fig. 12 demonstrate that LEGEND can significantly

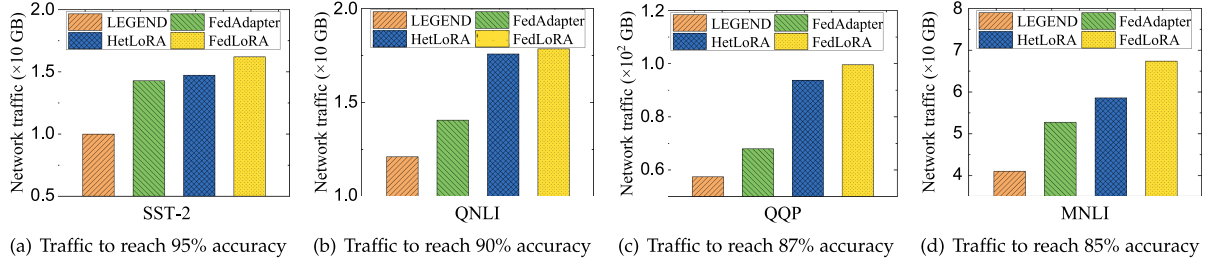


Fig. 10. Communication traffic of four approaches on general language understanding tasks.

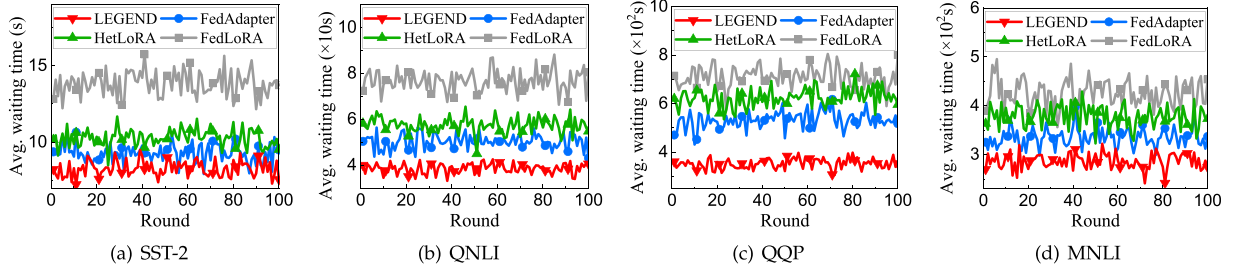


Fig. 11. Average waiting time of four approaches on general language understanding tasks.

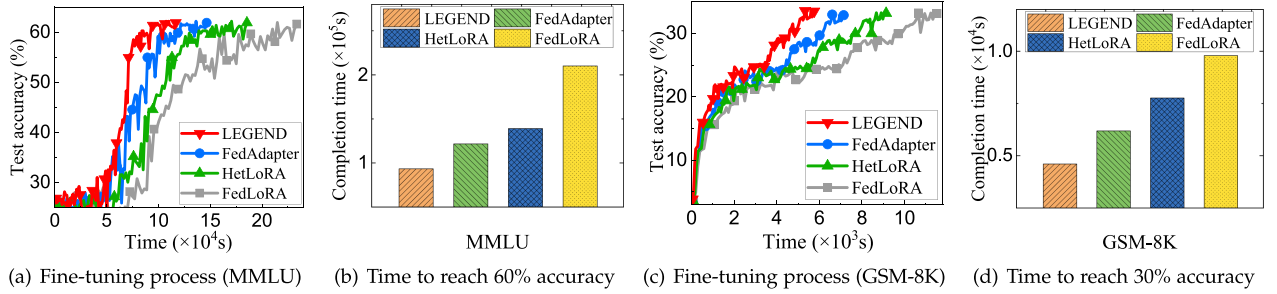


Fig. 12. Results on massive multitask understanding tasks and mathematical reasoning tasks.

accelerate the fine-tuning process and improve performance on massive multitask understanding and mathematical reasoning tasks. However, it is worth noting that as a LoRA-based parameter-efficient fine-tuning method, LEGEND, like other parameter-efficient approaches, demonstrates certain limitations when compared to full-parameter fine-tuning, particularly in more complex long-text generation and multi-step reasoning tasks [68], [69], [70]. These limitations stem from the inherent reduced tunable parameters and constrained optimization space of low-rank adaptation techniques, which may affect their ability to capture the intricate patterns required for sophisticated reasoning chains and coherent long-form generation. In future work, more advanced collaborative mechanisms, e.g., fine-tuning the entire model through model-parallel techniques [35], [71], will be needed to bridge this performance gap.

D. Ablation Study

There are two key factors in LEGEND, i.e., LoRA depth and rank distribution, which are developed to enhance the performance of FedLoRA. Herein, we conduct several sets of ablation experiments on SST-2 and QNLI to validate the effectiveness of the two essential factors. We adopt LEGEND, LEGEND

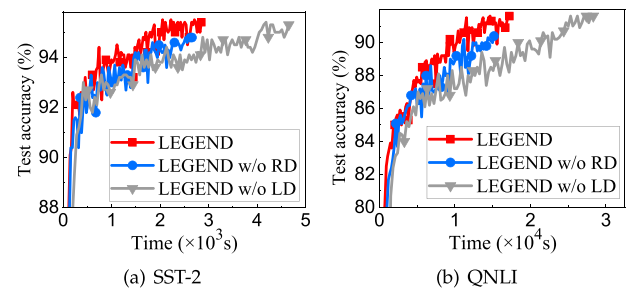


Fig. 13. Effect of LoRA depth and rank distribution.

without LoRA depth (denoted as LEGEND w/o LD), LEGEND without rank distribution (denoted as LEGEND w/o RD) as the baselines. As illustrated in Fig. 13, both LoRA depth and rank distribution are essential in LEGEND, but they contribute to the system in different ways. For instance, by Fig. 13(a) and (b), LEGEND w/o LD achieves similar final test accuracy, e.g., 95.3% on SST-2 and 91.6% on QNLI, compared with LEGEND, while LEGEND w/o RD slightly degrades to 94.6% on SST-2 and 90.5% on QNLI, respectively. Besides, by Fig. 13(a), when achieving a target accuracy of 94% on SST-2, LEGEND saves

the completion time by about 50.3% and 54.5% compared with LEGEND w/o RD and LEGEND w/o LD, respectively. By Fig. 13(b), compared with LEGEND w/o RD and LEGEND w/o LD, LEGEND separately speeds up fine-tuning by about $1.38\times$ and $1.93\times$, when achieving the same target accuracy (i.e., 90% on QNLI). On the one hand, by employing a reasonable rank distribution and enabling all devices to fine-tune all LoRA layers, LEGEND w/o LD enhances the fine-tuning performance, but results in prolonged waiting time and slow convergence rates. On the other hand, LEGEND w/o RD assigns adaptive LoRA depth for the devices with uniform ranks across all layers, failing to strategically allocate larger rank to critical layers (e.g., deep layers), which constrains the fine-tuning performance and leads to performance degradation. The results demonstrate the positive roles of the two designs.

VIII. RELATED WORKS

Natural Language Processing: The development of modern natural language processing (NLP) traces back to the introduction of the transformer architecture by Vaswani et al. in 2017 [1]. Upon transformer architecture, groundbreaking language models, such as BERT [3], GPT-2 [72], and more recently Llama [15], have achieved state-of-the-art results across various tasks. Specifically, the language model (LM) is first pre-trained on a large corpus to learn general features and patterns. Subsequently, the LM is further trained on domain-specific data generated on devices to enhance the model performance for a specific task [3], [5]. However, due to data privacy, it is impractical to collect enough data from the devices for centralized fine-tuning [6], [7].

Federated Learning: Federated Learning (FL) addresses the privacy concerns by enabling collaborative model training while keeping data decentralized [6], [73]. The foundational FedAvg algorithm allows devices to train models locally before global aggregation, establishing a privacy-preserving learning paradigm. For example, Chen et al. [74] explore the interplay between learning and communication efficiency in FL, while Wang et al. [54] provide theoretical convergence guarantees under heterogeneous conditions. Xia et al. [75] explore the applications of FL in healthcare domains. Recent efforts such as FedLamp [8], [9] aim to improve training efficiency in resource-constrained and heterogeneous edge environments by adapting local update frequency, model compression ratio, and batch size, yet they do not consider the unique challenges introduced by parameter-efficient fine-tuning techniques like LoRA in FL scenarios. Overall, these advances in general FL frameworks provide essential building blocks for emerging fine-tuning paradigms and domain-specific applications, particularly in NLP, where data privacy and heterogeneity challenges are pronounced.

Federated Fine-Tuning: Upon FL, Federated fine-tuning (FedFT) has been proposed to perform fine-tuning the pre-trained LM in a distributed manner [7]. However, the high resource costs associated with FedFT pose significant challenges to its practical implementation. Modern FedFT utilizes parameter-efficient fine-tuning (PEFT) methods, such as Adapter [11], [21], prompt-tuning [76], [77], to reduce on-device resources costs. For example, Cai et al. [11] first apply

Adapter in FedFT and propose FedAdapter, which dynamically searches for the optimal Adapter structure to improve the fine-tuning efficiency. However, due to the intrinsic property of Adapter, the fine-tuned Adapter-based LM inevitably brings additional inference latency, potentially resulting in up to a 30% latency increase [16], [22], [23], which is often unacceptable in practical applications. Upon prompt-tuning, Zhao et al. [77] propose FedPrompt to realize communication-efficient and privacy-preserving fine-tuning in federated setting. Nevertheless, prompt-tuning unavoidably occupies a portion of the model's input length, consequently diminishing the usable input space and extra inference latency.

Low-Rank Adaptation: To mitigate the disadvantages of the aforementioned PEFT methods, Hu et al. [16] propose low-rank adaptation (LoRA), which adds trainable rank decomposition matrices to each transformer layer of the LM while freezing the pre-trained weights of the LM to improve fine-tuning efficiency. LoRA achieves comparable performance of fully fine-tuning while introducing no additional inference latency, and has been widely adopted. For instance, Dettmers et al. [78] propose QLoRA, which is an efficient fine-tuning method based on model quantization, significantly reducing the memory requirement. Chen et al. [79] propose LongLoRA, which is an efficient fine-tuning approach to extend the context sizes of the LM with limited resource cost.

FedFT with LoRA: LoRA is naturally incorporated into FedFT to reduce resource costs. For example, Zhang et al. [20] propose FedLoRA and verify the efficiency of LoRA in the context of FedFT through extensive experiments. Upon FedLoRA, Cho et al. [27] propose HetLoRA, in which each device adds LoRA layers to all transformer layers with a diverse and appropriate LoRA rank to deal with system heterogeneity. However, due to the rank mismatch of all LoRA layers on different devices, it is difficult to aggregate these layers, resulting in poor fine-tuning performance. Recent work has explored fine-tuning generative models such as Stable Diffusion [80] for AIGC tasks, demonstrating benefits in communication efficiency and privacy protection. However, unlike our work, these works do not address the challenge of efficient LoRA-based fine-tuning under system heterogeneity and resource constraints. LEGEND generates a reasonable rank distribution to enhance the fine-tuning performance and assigns different LoRA depths for heterogeneous devices to improve fine-tuning efficiency, effectively addressing the aforementioned challenges while saving resource costs.

IX. CONCLUSION

In this paper, we review the intrinsic properties of FedFT and propose an efficient LoRA-based FedFT framework, called LEGEND, to address resource constraints and system heterogeneity. We analyze the coupled relationship between LoRA depth and rank distribution, and design an efficient LoRA configuration algorithm for heterogeneous devices, thereby promoting fine-tuning efficiency. Extensive experiments are conducted on a real platform of 80 wireless devices. The experimental results show that LEGEND significantly outperforms the existing methods, providing a speedup of $1.5\text{--}2.8\times$ and saving communication costs by about 42.3% while achieving the target accuracy.

REFERENCES

- [1] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.
- [2] H. Bai et al., "BinaryBERT: Pushing the limit of BERT quantization," 2020, *arXiv: 2012.15701*.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv: 1810.04805*.
- [4] L. Hou, Z. Huang, L. Shang, X. Jiang, X. Chen, and Q. Liu, "DynaBERT: Dynamic BERT with adaptive width and depth," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 9782–9793.
- [5] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
- [6] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int. Conf. Artif. Intell. Statist.*, PMLR, 2017, pp. 1273–1282.
- [7] B. Y. Lin et al., "FedNLP: Benchmarking federated learning methods for natural language processing tasks," 2021, *arXiv:2104.08815*.
- [8] Y. Xu, Y. Liao, H. Xu, Z. Ma, L. Wang, and J. Liu, "Adaptive control of local updating and model compression for efficient federated learning," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 5675–5689, Oct. 2023.
- [9] Y. Liao, Y. Xu, H. Xu, Z. Yao, L. Wang, and C. Qiao, "Accelerating federated learning with data and model parallelism in edge computing," *IEEE/ACM Trans. Netw.*, vol. 32, no. 1, pp. 904–918, Feb. 2024.
- [10] J. Stremmel and A. Singh, "Pretraining federated text models for next word prediction," in *Proc. Future Inf. Commun. Conf.*, Springer, 2021, pp. 477–488.
- [11] D. Cai, Y. Wu, S. Wang, F. X. Lin, and M. Xu, "FedAdapter: Efficient federated learning for modern NLP," 2022, *arXiv:2205.10162*.
- [12] L. Zhu et al., "PockEngine: Sparse and efficient fine-tuning in a pocket," in *Proc. 56th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2023, pp. 1381–1394.
- [13] S. Dhar, J. Guo, J. Liu, S. Tripathi, U. Kurup, and M. Shah, "A survey of on-device machine learning: An algorithms and learning theory perspective," *ACM Trans. Internet of Things*, vol. 2, no. 3, pp. 1–49, 2021.
- [14] Y. Liao, Y. Xu, H. Xu, L. Wang, Z. Yao, and C. Qiao, "MergeSFL: Split federated learning with feature merging and batch size regulation," in *Proc. IEEE 40th Int. Conf. Data Eng.*, 2024, pp. 2054–2067.
- [15] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
- [16] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," 2021, *arXiv:2106.09685*.
- [17] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "Oort: Efficient federated learning via guided participant selection," in *Proc. 15th USENIX Symp. Operating Syst. Des. Implementation*, 2021, pp. 19–35.
- [18] J. Liu, J. Liu, H. Xu, Y. Liao, Z. Wang, and Q. Ma, "YOGA: Adaptive layer-wise model aggregation for decentralized federated learning," *IEEE/ACM Trans. Netw.*, vol. 32, no. 2, pp. 1768–1780, Apr. 2024.
- [19] Y. Liao, Y. Xu, H. Xu, L. Wang, and C. Qian, "Adaptive configuration for heterogeneous participants in decentralized federated learning," in *Proc. IEEE Conf. Comput. Commun.*, 2023, pp. 1–10.
- [20] Z. Zhang et al., "FedPETuning: When federated learning meets the parameter-efficient tuning methods of pre-trained language models," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2023, pp. 9963–9977.
- [21] N. Houlsby et al., "Parameter-efficient transfer learning for NLP," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2019, pp. 2790–2799.
- [22] B. Liao, Y. Meng, and C. Monz, "Parameter-efficient fine-tuning without introducing new latency," 2023, *arXiv:2305.16742*.
- [23] S. Li et al., "CaraServe: CPU-assisted and rank-aware LoRA serving for generative LLM inference," 2024, *arXiv:2401.11240*.
- [24] M. Wankhade, A. C. S. Rao, and C. Kulkarni, "A survey on sentiment analysis methods, applications, and challenges," *Artif. Intell. Rev.*, vol. 55, no. 7, pp. 5731–5780, 2022.
- [25] S. Minaee, N. Kalchbrenner, E. Cambria, N. Nikzad, M. Chenaghlu, and J. Gao, "Deep learning–based text classification: A comprehensive review," *ACM Comput. Surv.*, vol. 54, no. 3, pp. 1–40, 2021.
- [26] Q. Zhang et al., "Adaptive budget allocation for parameter-efficient fine-tuning," in *Proc. 11th Int. Conf. Learn. Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=lq62uWRJjY>
- [27] Y. J. Cho, L. Liu, Z. Xu, A. Fahrezi, M. Barnes, and G. Joshi, "Heterogeneous LoRA for federated fine-tuning of on-device foundation models," in *Proc. Int. Workshop Federated Learn. Age Found. Models Conjunction NeurIPS*, 2024, pp. 12903–12913.
- [28] Y. G. Kim and C.-J. Wu, "AutoFL: Enabling heterogeneity-aware energy efficient federated learning," in *Proc. 54th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2021, pp. 183–198.
- [29] B. Luo, W. Xiao, S. Wang, J. Huang, and L. Tassioulas, "Tackling system and statistical heterogeneity for federated learning with adaptive client sampling," in *Proc. 2022 IEEE Conf. Comput. Commun.*, 2022, pp. 1739–1748.
- [30] Y. Liao, Y. Xu, H. Xu, L. Wang, Z. Yao, and C. Qiao, "MergeSFL: Split federated learning with feature merging and batch size regulation," 2023, *arXiv:2311.13348*.
- [31] Wikimedia Foundation, "Wikimedia downloads," (n.d.) Accessed: Jul. 22, 2025. [Online]. Available: <https://dumps.wikimedia.org>
- [32] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," 2019, *arXiv: 1910.10683*.
- [33] M. Xu, Y. Wu, D. Cai, X. Li, and S. Wang, "Federated fine-tuning of billion-sized language models across mobile devices," 2023, *arXiv:2308.13894*.
- [34] I. Loshchilov, "Decoupled weight decay regularization," 2017, *arXiv: 1711.05101*.
- [35] J. Kaddour, J. Harris, M. Mozes, H. Bradley, R. Raileanu, and R. McHardy, "Challenges and applications of large language models," 2023, *arXiv:2307.10169*.
- [36] C. Wei, Y. Shu, Y. T. He, and F. R. Yu, "Flexora: Flexible low rank adaptation for large language models," 2024, *arXiv:2408.10774*.
- [37] Y. Guo, H. Shi, A. Kumar, K. Grauman, T. Rosing, and R. Feris, "SpotTune: Transfer learning through adaptive fine-tuning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 4805–4814.
- [38] Q. Zhang et al., "Adaptive budget allocation for parameter-efficient fine-tuning," 2023, *arXiv:2303.10512*.
- [39] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv: 1907.11692*.
- [40] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," 2018, *arXiv: 1804.07461*.
- [41] X. Ma, J. Zhang, S. Guo, and W. Xu, "Layer-wised model aggregation for personalized federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10092–10101.
- [42] H. Song, H. Zhao, S. Majumder, and T. Lin, "Increasing model capacity for free: A simple strategy for parameter efficient fine-tuning," 2024, *arXiv:2407.01320*.
- [43] R. Shuttlesworth, J. Andreas, A. Torralba, and P. Sharma, "LoRA vs full fine-tuning: An illusion of equivalence," 2024, *arXiv:2410.21228*.
- [44] C. Gao et al., "Higher layers need more LoRA experts," 2024, *arXiv:2402.08562*.
- [45] J. Konecny, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," 2016, *arXiv:1610.05492*.
- [46] D. Leroy, A. Coucke, T. Lavril, T. Gisselbrecht, and J. Dureau, "Federated learning for keyword spotting," in *Proc. 2019 IEEE Int. Conf. Acoust. Speech Signal Process.*, 2019, pp. 6341–6345.
- [47] D. Halperin, W. Hu, A. Sheth, and D. Wetherall, "Predictable 802.11 packet delivery from wireless channel measurements," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 40, no. 4, pp. 159–170, 2010.
- [48] C. Yue, R. Jin, K. Suh, Y. Qin, B. Wang, and W. Wei, "LinkForecast: Cellular link bandwidth prediction in LTE networks," *IEEE Trans. Mobile Comput.*, vol. 17, no. 7, pp. 1582–1594, Jul. 2018.
- [49] H. Qi, E. R. Sparks, and A. Talwalkar, "Paleo: A performance model for deep neural networks," in *Proc. Int. Conf. Learn. Representations*, 2017. [Online]. Available: <https://openreview.net/forum?id=SyVVJ85lg>
- [50] S. Li et al., "PyTorch distributed: Experiences on accelerating data parallel training," 2020, *arXiv: 2006.15704*.
- [51] D. Narayanan et al., "Efficient large-scale language model training on GPU clusters using megatron-LM," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2021, pp. 1–15.
- [52] R. W. Cottle, "Nonlinear programs with positively bounded Jacobians," *SIAM J. Appl. Math.*, vol. 14, no. 1, pp. 147–158, 1966.
- [53] G. H. Golub, A. Hoffman, and G. W. Stewart, "A generalization of the Eckart-Young-Mirsky matrix approximation theorem," *Linear Algebra Appl.*, vol. 88, pp. 317–327, 1987.
- [54] K. Wang, Y. Ma, M. B. Mashhadi, C. H. Foh, R. Tafazolli, and Z. Ding, "Convergence acceleration in wireless federated learning: A Stackelberg game approach," *IEEE Trans. Veh. Technol.*, vol. 74, no. 1, pp. 714–729, Jan. 2025.
- [55] Z. Wang, Y. Zhou, Y. Shi, and K. B. Letaief, "Federated fine-tuning for pre-trained foundation models over wireless networks," *IEEE Trans. Wireless Commun.*, vol. 24, no. 4, pp. 3450–3464, Apr. 2025.
- [56] Z. Zhang, Y. Yang, Y. Dai, L. Qu, and Z. Xu, "When federated learning meets pre-trained language models' parameter-efficient tuning methods," 2022, *arXiv:2212.10025*.

- [57] S. Mittal, "A survey on optimized implementation of deep learning models on the NVIDIA Jetson platform," *J. Syst. Archit.*, vol. 97, pp. 428–442, 2019.
- [58] D. Merkel et al., "Docker: Lightweight linux containers for consistent development and deployment," *Linux J.*, vol. 2014, no. 239, 2014, Art. no. 2.
- [59] N. Naik, "Building a virtual system of systems using docker swarm in multiple clouds," in *Proc. 2016 IEEE Int. Symp. Syst. Eng.*, 2016, pp. 1–3.
- [60] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 8026–8037.
- [61] E. Gabriel et al., "Open MPI: Goals, concept, and design of a next generation MPI implementation," in *Proc. 11th Eur. Parallel Virtual Mach. Message Passing Interface Users' Group Meeting*, Budapest, Hungary, Springer, 2004, pp. 97–104.
- [62] IPerf: The TCP/UDP bandwidth measurement tool, 1999. [Online]. Available: <http://dast.nlanr.net/Projects/Iperf/>
- [63] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empir. Methods Natural Lang. Process.: Syst. Demonstrations*, 2020, pp. 38–45.
- [64] P. He, J. Gao, and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing," 2021, *arXiv:2111.09543*.
- [65] D. Hendrycks et al., "Measuring massive multitask language understanding," 2020, *arXiv: 2009.03300*.
- [66] Y. Wang, X. Liu, and S. Shi, "Deep neural solver for math word problems," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2017, pp. 845–854.
- [67] K. Cobbe et al., "Training verifiers to solve math word problems," 2021, *arXiv:2110.14168*.
- [68] D. Biderman et al., "LoRA learns less and forgets less," 2024, *arXiv:2405.09673*.
- [69] E. Lobo, C. Agarwal, and H. Lakkaraju, "On the impact of fine-tuning on chain-of-thought reasoning," 2024, *arXiv:2411.15382*.
- [70] Y. Mao et al., "A survey on LoRA of large language models," *Front. Comput. Sci.*, vol. 19, no. 7, 2025, Art. no. 197605.
- [71] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, "SplitFed: When federated learning meets split learning," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 8485–8493.
- [72] A. Radford et al., "Language models are unsupervised multitask learners," 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID:160025533>
- [73] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1/2, pp. 1–210, 2021.
- [74] M. Chen, Z. Yang, W. Saad, C. Yin, H. V. Poor, and S. Cui, "A joint learning and communications framework for federated learning over wireless networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 1, pp. 269–283, Jan. 2021.
- [75] W. Xia, W. Wen, K.-K. Wong, T. Q. Quek, J. Zhang, and H. Zhu, "Federated-learning-based client scheduling for low-latency wireless communications," *IEEE Wireless Commun.*, vol. 28, no. 2, pp. 32–38, Apr. 2021.
- [76] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," 2021, *arXiv:2104.08691*.
- [77] H. Zhao, W. Du, F. Li, P. Li, and G. Liu, "FedPrompt: Communication-efficient and privacy-preserving prompt tuning in federated learning," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2023, pp. 1–5.
- [78] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 10088–10115.
- [79] Y. Chen et al., "LongLoRA: Efficient fine-tuning of long-context large language models," 2023, *arXiv:2309.12307*.
- [80] X. Huang et al., "Federated learning-empowered AI-generated content in wireless networks," *IEEE Netw.*, vol. 38, no. 5, pp. 304–313, Sep. 2024.



Jun Liu received the BS degree from Central South University, in 2019. He is currently working toward the PhD degree with the School of Computer Science and Technology, University of Science and Technology of China (USTC). His main research interests are edge computing and federated learning.



Yunming Liao received the BS degree from the University of Science and Technology of China, in 2020. He is currently working toward the PhD degree with the School of Computer Science and Technology, University of Science and Technology of China. His research interests include mobile edge computing and federated learning.



Hongli Xu (Member, IEEE) received the BS degree in computer science from the University of Science and Technology of China, China, in 2002, and the PhD degree in computer software and theory from the University of Science and Technology of China, China, in 2007. He is a professor with the School of Computer Science and Technology, University of Science and Technology of China (USTC), China. He was awarded the Outstanding Youth Science Foundation of NSFC, in 2018. He has won the best paper award or the best paper candidate in several famous

conferences. He has published more than 100 papers in famous journals and conferences, including *IEEE/ACM Transactions on Networking*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Parallel and Distributed Systems*, *Infocom* and *ICNP*, etc. He has also held more than 30 patents. His main research interest is software defined networks, edge computing and Internet of Things.



Yang Xu (Member, IEEE) received the BS degree from the Wuhan University of Technology, in 2014, and the PhD degree in computer science and technology from the University of Science and Technology of China, in 2019. He is currently an associate researcher with the School of Computer Science and Technology, University of Science and Technology of China. His primary research interests include the Internet of Things, edge computing, federated learning, and edge intelligence.



Jianchun Liu (Member, IEEE) received the PhD degree from the School of Data Science, University of Science and Technology of China, in 2022. He is currently an associate researcher with the School of Computer Science and Technology, University of Science and Technology of China. His main research interests are computer networks, edge computing and federated learning. He is a member of the ACM.



Chen Qian (Senior Member, IEEE) received the BS degree from Nanjing University, in 2006, the MPhil degree from The Hong Kong University of Science and Technology, in 2008, and the PhD degree from The University of Texas at Austin, in 2013, all in computer science. He is currently an assistant professor with the Department of Computer Engineering, University of California at Santa Cruz. His research interests include computer networking, network security, and Internet of Things. He has authored more than 60 research papers in highly competitive conferences and journals. He is a member of the ACM.